Programmierung 2
Arbeitsheft

Von null bis zur Klausuraufgabe

PIB-PR2 · Prof. Dr. Markus Esch · HTW Saar

Klausur: 20. August 2026 · 120 Minuten · ohne Hilfsmittel

So ist dieses Heft aufgebaut

Die Module stehen in Abhängigkeitsreihenfolge: Nichts kommt vor, was nicht vorher erklärt wurde. Du kannst von Seite eins an durcharbeiten, ohne vorwärts blättern zu müssen.

Unter jeder Aufgabe steht in grauer Schrift, welche Abschnitte sie voraussetzt. Wenn du irgendwo hängst, weißt du sofort, wohin du zurückspringen musst.

Jedes Modul beginnt mit einem Kasten „Baut auf / Wird gebraucht für" – damit siehst du, wo du gerade stehst und wofür der Stoff später gut ist.

Vier Stufen pro Thema:

Stufe 1 Verstehen – ankreuzen, ausfüllen, kurze Fragen.
Stufe 2 Einzelteile – eine Methode, ein Lambda. Passt in einen 25-Minuten-Block.
Stufe 3 Zusammensetzen – eine ganze Klasse.
Stufe 4 Klausurniveau – eine vollständige Prüfungsaufgabe.

Schreib von Hand in die grauen Kästen. In der Klausur gibt es keine IDE. Lösungen stehen hinten in Teil Z.

Alle Lösungen wurden mit OpenJDK 21 bzw. gcc 13 ausgeführt und verifiziert.

Inhaltsverzeichnis

Die Landkarte

So hängen die Module zusammen. Die rechte Spalte zeigt, warum das Modul überhaupt drankommt.

Modul	Thema	Baut auf	Klausurrelevanz
1	Variablen, Arrays, Schleifen	–	Grundlage für alles
2	Klassen und Objekte	1	Grundlage für Aufgabe 3
3	Vererbung und abstrakte Klassen	2	Aufgabe 3, Teil a und b
4	ArrayList	2	Collections +++
5	HashMap, HashSet, equals	4	Collections +++
6	Interfaces und Comparable	3	Aufgabe 3, fast immer verlangt
7	Lambda und funktionale Interfaces	6	Lambda +++
8	Generics	3, 4	Generics +++
9	Die Verwalterklasse	3–8	Aufgabe 3, Teil c
10	Collection auswählen	5	Übung 7 und 8, Begründungsfrage
11	Streams	7, 10	Aufgabe 4
12	Innere Klassen	2, 7	Innere Klassen +
13	Rekursion	1, 2	Rekursion +
14	C: Grundlagen und Pointer	1	Pointer ++
15	C: Structs, Arrays, Matrizen	14	C-Aggregattypen ++
16	Was gibt der Code aus?	1–15	Aufgabe 2
17	Probeklausur	alles	Generalprobe

Wenn die Zeit knapp wird

Pflicht sind Modul 1 bis 9 – sie führen direkt zur größten Klausuraufgabe. Danach in dieser Reihenfolge: 11 (Streams), 14 (Pointer), 16 (Trainer), 10, 15.

Streichbar sind zuerst Modul 12 (Innere Klassen) und 13 (Rekursion). Beide haben nur Gewicht +.

Modul 9 solltest du am Ende zweimal geschrieben haben – beim zweiten Mal ohne Nachschlagen. Das ist der beste einzelne Zeitinvest für diese Klausur.

Arbeitsplan

Tag	Block 1 (25 min)	Block 2 (25 min)
1	Modul 1 und 2	Modul 2 zu Ende
2	Modul 3	Modul 3 Stufe 3
3	Modul 4	Modul 5
4	Modul 6	Modul 7 Stufe 1 und 2
5	Modul 7 Stufe 3, Modul 8	Modul 9 Stufe 1 bis 3
6	`Modul 9 Stufe 4 (Klausuraufgabe)`	Modul 10 und 11 Stufe 1 bis 2
7	Modul 11 Stufe 3 und 4	Modul 14
8	Modul 15	Modul 12 und 13
9	Modul 16 Trainer	Modul 9 Stufe 4 nochmal, ohne Vorlage
10	Probeklausur Teil 1 und 2	Probeklausur Teil 3 und 4

Modul 1 – Variablen, Arrays, Schleifen

Baut auf: nichts

Wird gebraucht für: wirklich alles Weitere

1.1 Variablen

```
int     zahl  = 5;          // ganze Zahl
double  komma = 3.5;        // Kommazahl
String  text  = "Hallo";    // Text in doppelten Anfuehrungszeichen
boolean ja    = true;       // wahr oder falsch
```

String beginnt groß, die anderen klein. Das ist kein Zufall: int, double und boolean sind primitive Typen, String ist eine Klasse. Der Unterschied wird in Modul 4 wichtig – in Listen dürfen nur Klassentypen.

1.2 Arrays

```
int[] zahlen = new int[3];    // drei Faecher: Index 0, 1, 2
zahlen[0] = 10;
zahlen[1] = 20;
zahlen[2] = 30;

int[] direkt = {10, 20, 30};  // kuerzer, gleich befuellt

direkt.length                 // 3    Anzahl, OHNE Klammern
direkt[0]                     // 10   erstes Element
direkt[2]                     // 30   letztes Element
```

Die häufigste Falle

Bei Länge 3 ist der letzte gültige Index die 2. Eine Schleife läuft deshalb immer bis kleiner length, nie bis kleiner gleich.

Und drei Schreibweisen für dasselbe: Array hat length ohne Klammern, String hat length() mit Klammern, Liste hat size(). Das verwechselt jeder.

1.3 Schleifen

```
// klassisch: wenn du den Index brauchst
for (int i = 0; i < direkt.length; i++) {
    System.out.print(direkt[i] + " ");
}
// Ausgabe: 10 20 30

// for-each: wenn du nur die Werte brauchst
for (int x : direkt) {
    System.out.print(x + " ");
}
// Ausgabe: 10 20 30
```

for-each lesen

for (int x : direkt) heißt „für jedes int x aus direkt". Links der Typ eines einzelnen Elements plus ein frei gewählter Name, rechts die Sammlung.

Nimm for-each immer, wenn du den Index nicht brauchst. Fast alle Beispiele in diesem Heft benutzen es.

Aufgabe 1.1 Stufe 1 Array anlegen und ausgeben

Voraussetzung: *1.2 und 1.3*

Lege ein Array mit den Zahlen 4, 8, 15 an und gib alle Werte mit einer for-each-Schleife aus.

Deine Lösung:

Aufgabe 1.2 Stufe 2 Mit Index arbeiten

Voraussetzung: *1.2 und 1.3*

Schreibe eine Methode int summe(int[] a), die alle Werte eines Arrays addiert und die Summe zurückgibt. Schreibe danach int maximum(int[] a).

Deine Lösung:

Modul 2 – Klassen und Objekte

Baut auf: Modul 1

Wird gebraucht für: Modul 3 bis 9, also die gesamte Klausuraufgabe 3

2.1 Bauplan und Ding

Eine Klasse ist ein Bauplan, ein Objekt ein daraus gebautes Ding. Der Bauplan Hund beschreibt, dass jeder Hund einen Namen und ein Alter hat. Bello ist ein konkretes Objekt.

```
class Hund {
    private String name;                  // 1. FELDER: was ein Hund hat
    private int alter;

    public Hund(String name, int alter) { // 2. KONSTRUKTOR: wie man einen baut
        this.name  = name;
        this.alter = alter;
    }

    public String getName()  { return name; }   // 3. GETTER: Werte nach aussen
    public int    getAlter() { return alter; }

    @Override public String toString() {        // 4. TOSTRING: wie er sich ausgibt
        return name + " (" + alter + ")";
    }
}
```

Benutzung

```
Hund h = new Hund("Bello", 3);
h.getName()                 // Bello
h.getAlter()                // 3
System.out.println(h);      // Bello (3)     <- ruft toString auf
```

Die drei Stellen, die verwirren

private heißt: von außen kommt niemand direkt an das Feld. h.name gäbe einen Fehler. Die Getter sind die kontrollierte Tür.

this.name = name – Parameter und Feld heißen gleich. this.name meint das Feld des Objekts, name allein den Parameter. Übersetzt: „nimm den übergebenen Wert und schreib ihn in mein Feld".

Der Konstruktor heißt genau wie die Klasse und hat keinen Rückgabetyp – kein void, gar nichts. Das ist die einzige Stelle in Java, wo das so ist.

Aufgabe 2.1 Stufe 1 Begriffe zuordnen

Voraussetzung: *2.1*

Begriff	Buchstabe	Erklärung
Feld	____	A – Methode, die einen Feldwert nach außen gibt
Konstruktor	____	B – Variable, die zu einem Objekt gehört
Getter	____	C – legt fest, wie das Objekt als Text aussieht
toString	____	D – Methode, die beim Erzeugen aufgerufen wird
private	____	E – nur innerhalb der Klasse zugänglich

Aufgabe 2.2 Stufe 2 Eine Klasse schreiben

Voraussetzung: *2.1*

Schreibe eine Klasse Buch mit den privaten Feldern titel (String) und seiten (int). Ergänze Konstruktor, beide Getter und toString, das etwa "Faust (280 Seiten)" ausgibt.

Deine Lösung:

Aufgabe 2.3 Stufe 2 Fehler finden

Voraussetzung: *2.1*

In jedem Ausschnitt steckt genau ein Fehler.

a)

```
class Auto {
    private String marke;
    public void Auto(String marke) { this.marke = marke; }
}
```

Fehler: _____

b)

```
class Auto {
    private String marke;
    public Auto(String marke) { marke = marke; }
}
```

Fehler: _____

c)

```
Auto a = new Auto("VW");
System.out.println(a.marke);      // marke ist private
```

Fehler: _____

Modul 3 – Vererbung und abstrakte Klassen

Baut auf: Modul 2

Wird gebraucht für: Modul 6, 8, 9 – und Teil a und b jeder Klausuraufgabe 3

3.1 Warum Vererbung

Hund und Katze haben beide einen Namen und ein Alter. Statt das zweimal zu schreiben, zieht man das Gemeinsame in eine Oberklasse Tier. Hund und Katze erben es.

Die Oberklasse ist abstract, weil es „ein Tier" als solches nicht gibt – nur konkrete Hunde und Katzen. Von einer abstrakten Klasse kann man keine Objekte erzeugen.

```
abstract class Tier {
    private final String name;
    private final int alter;

    public Tier(String name, int alter) {
        this.name  = name;
        this.alter = alter;
    }
    public String getName()  { return name; }
    public int    getAlter() { return alter; }

    public abstract String laut();      // OHNE Rumpf, endet mit Semikolon

    @Override public String toString() { return name + " (" + alter + ")"; }
}

class Hund extends Tier {
    private final String rasse;                  // eigenes Feld

    public Hund(String name, int alter, String rasse) {
        super(name, alter);                      // ZUERST: Oberklasse befuellen
        this.rasse = rasse;                      // dann das eigene Feld
    }
    public String getRasse() { return rasse; }

    @Override public String laut() { return "Wau"; }   // Pflicht!
}
```

Die vier Regeln

- super(...) muss die ERSTE Anweisung im Konstruktor sein.
- Auf private Felder der Oberklasse kommst du nur über die Getter, nie direkt.
- Jede abstrakte Methode MUSS in jeder konkreten Unterklasse implementiert werden – sonst muss die Unterklasse selbst abstract sein.
- Eine Klasse kann nur EINE Klasse erweitern. Mehrfachvererbung gibt es in Java nicht.

3.2 Polymorphie

```
Tier t = new Hund("Rex", 5, "Boxer");   // Variablentyp Tier, Objekttyp Hund
t.laut();          // "Wau"  -> es zaehlt der OBJEKTtyp
// t.getRasse();   // FEHLER -> es zaehlt der VARIABLENtyp

if (t instanceof Hund h) {        // Pruefen und umwandeln in einem Schritt
    System.out.println(h.getRasse());   // Boxer
}
```

```
Tier[] tiere = { new Hund("Bello",3,"Dackel"), new Katze("Minka",2,true) };
for (Tier x : tiere) System.out.println(x.getName() + ": " + x.laut());
// Bello: Wau
// Minka: Miau
```

Der Kern der Polymorphie

Welche Methode ausgeführt wird, entscheidet das Objekt zur Laufzeit – nicht der Typ der Variablen. Deshalb liefert dieselbe Schleife bei jedem Element eine andere Ausgabe.

Welche Methoden du überhaupt aufrufen darfst, entscheidet dagegen der Variablentyp. Deshalb geht getRasse nicht, solange die Variable Tier heißt.

Aufgabe 3.1 Stufe 1 Wahr oder falsch

Voraussetzung: *3.1*

- ☐ Von einer abstrakten Klasse kann man Objekte erzeugen.
- ☐ Eine abstrakte Klasse kann einen Konstruktor haben.
- ☐ super(…) muss die erste Anweisung im Konstruktor sein.
- ☐ Eine Unterklasse kann auf private Felder der Oberklasse direkt zugreifen.
- ☐ Jede abstrakte Methode muss in jeder konkreten Unterklasse implementiert werden.
- ☐ Eine Klasse kann mehrere Klassen gleichzeitig erweitern.
- ☐ Eine abstrakte Methode hat einen Rumpf.

Aufgabe 3.2 Stufe 1 Lücken füllen

Voraussetzung: *3.1*

```
___________ class Fahrzeug {
    ___________ String name;              // von aussen nicht zugreifbar

    public Fahrzeug(String name) {
        ___________.name = name;
    }
    public ___________ double steuer();   // ohne Rumpf
}

class PKW ___________ Fahrzeug {
    private int sitze;
    public PKW(String name, int sitze) {
        ___________(name);
        this.sitze = sitze;
    }

    ___________
    public double steuer() { return sitze * 20.0; }
}
```

Aufgabe 3.3 Stufe 2 Abstrakte Oberklasse schreiben

Voraussetzung: *3.1*

Schreibe eine abstrakte Klasse Tier mit den privaten Feldern name (String) und alter (int). Ergänze Konstruktor, beide Getter, eine abstrakte Methode String laut() und toString.

Deine Lösung:

Aufgabe 3.4 Stufe 2 Zwei Unterklassen schreiben

Voraussetzung: *3.3*

Schreibe Hund mit dem zusätzlichen Feld rasse (String) und Katze mit dem zusätzlichen Feld stubentiger (boolean). Beide brauchen Konstruktor, Getter für das eigene Feld und eine Implementierung von laut().

Deine Lösung:

Aufgabe 3.5 Stufe 2 Fehler finden

Voraussetzung: *3.1*

a)

```
abstract class Form {
    public abstract double flaeche() { return 0; }
}
```

Fehler: _____

b)

```
class Kreis extends Form {
    private int radius;
    public Kreis(int radius) {
        this.radius = radius;
        super("Kreis");
    }
}
```

Fehler: _____

c)

```
abstract class Tier { private String name; }
class Hund extends Tier {
    public String beschreibe() { return "Hund: " + name; }
}
```

Fehler: _____

d)

```
abstract class Tier { public abstract String laut(); }
class Fisch extends Tier {
    private String art;
    public Fisch(String art) { this.art = art; }
}
```

Fehler: _____

Aufgabe 3.6 Stufe 3 UML-Diagramm umsetzen

Voraussetzung: *3.1 und 3.2*

Gegeben ist folgendes Diagramm in Textform

```
abstract Konto
  - inhaber : String        (private)
  - stand   : double        (private)
  + Konto(inhaber, stand)
  + getInhaber() : String
  + getStand()   : double
  + einzahlen(betrag : double) : void
  + zinsen() : double        ABSTRAKT
  + toString() : String

Girokonto extends Konto
  - dispo : double
  + zinsen() -> 0.0

Sparkonto extends Konto
  - zinssatz : double
  + zinsen() -> stand * zinssatz
```

Setze das Diagramm vollständig in Java um.

Deine Lösung:

Modul 4 – ArrayList

Baut auf: Modul 2

Wird gebraucht für: Modul 5, 8, 9, 10, 11 – jede Verwalterklasse

4.1 Warum nicht einfach ein Array

Ein Array hat eine feste Größe. new int[3] fasst genau drei Werte – für immer. Willst du einen vierten, musst du ein neues Array bauen und umkopieren.

Eine ArrayList wächst von selbst. Genau deshalb verlangte Übung 4 in deinem Semester, die Arrays in Depot und Silo durch eine Liste zu ersetzen.

```
import java.util.*;                    // ganz oben in der Datei noetig

ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

System.out.println(namen);             // [Anna, Ben, Clara]
```

Die spitzen Klammern

ArrayList<String> heißt: eine Liste, in der Strings liegen. In den spitzen Klammern steht der Typ der Elemente.

Links schreibst du ihn aus, rechts reichen leere Klammern: new ArrayList<>(). Java ergänzt den Typ selbst – das heißt Diamond-Operator.

Dort darf nur ein Klassentyp stehen. ArrayList<int> ist verboten, ArrayList<Integer> richtig. Integer ist die Klassen-Variante von int.

4.2 Die Methoden

Methode	Was sie tut	Beispiel
`add(x)`	hängt hinten an	`namen.add("Eva")`
`size()`	Anzahl	`namen.size()` → `3`
`get(i)`	Element an Position i	`namen.get(1)` → `Ben`
`contains(x)`	ist x drin?	`namen.contains("Ben")` → `true`
`remove(x)`	entfernt x	`namen.remove("Ben")`
`isEmpty()`	ist sie leer?	`namen.isEmpty()` → `false`

Darüber laufen und mit eigenen Objekten

```
for (String n : namen) System.out.print(n + " ");

ArrayList<Hund> hunde = new ArrayList<>();
hunde.add(new Hund("Bello", 3, "Dackel"));
hunde.add(new Hund("Rex", 5, "Boxer"));

for (Hund h : hunde) System.out.print(h.getName() + " ");
// Ausgabe: Bello Rex
```

Array und Liste nebeneinander

```
Array:      a.length       a[0]           feste Groesse
ArrayList:  liste.size()   liste.get(0)   waechst mit
```

Aufgabe 4.1 Stufe 1 Ausgaben vorhersagen

Voraussetzung: *4.1 und 4.2*

```
ArrayList<String> l = new ArrayList<>();
l.add("a"); l.add("b"); l.add("a");
```

l.size() _____

l.get(0) _____

l.contains("b") _____

l nach l.remove("a") _____

Aufgabe 4.2 Stufe 2 ArrayList benutzen

Voraussetzung: *Modul 2, Abschnitt 4.1 und 4.2*

Lege eine ArrayList für Bücher an, füge zwei Bücher hinzu, gib mit einer for-each-Schleife alle Titel aus und danach die Anzahl.

Deine Lösung:

Aufgabe 4.3 Stufe 2 Liste durchsuchen

Voraussetzung: *4.2*

Schreibe eine Methode Buch findeNachTitel(ArrayList<Buch> liste, String titel). Sie geht die Liste durch und gibt das passende Buch zurück, oder null, wenn keins passt.

Denk daran: Strings vergleicht man mit equals, nicht mit ==.

Deine Lösung:

Modul 5 – HashMap, HashSet, equals und hashCode

Baut auf: Modul 4

Wird gebraucht für: Modul 9, 10, 11 – und die Verwalterklasse in Aufgabe 3

5.1 HashMap: nachschlagen statt suchen

Das Problem, das sie löst

500 Hunde in einer ArrayList, gesucht ist „Bello": Du musst die Liste von vorne durchgehen, im schlimmsten Fall 500 Vergleiche.

Eine HashMap ist wie ein Telefonbuch: Du schlägst beim Namen nach und bist sofort da, egal wie groß sie ist. Genau deshalb stand in Übung 7 die Anforderung „effizientes Suchen anhand der Produkt-ID".

```
HashMap<String, Integer> alter = new HashMap<>();
//         ^Schluessel  ^Wert

alter.put("Anna", 21);
alter.put("Ben",  23);

alter.get("Anna")             // 21
alter.get("Zoe")              // null    <- nicht vorhanden!
alter.getOrDefault("Zoe", 0)  // 0       <- mit Ersatzwert
alter.containsKey("Ben")      // true
alter.size()                  // 2

for (String k : alter.keySet())
    System.out.println(k + " = " + alter.get(k));
```

Drei Dinge, die überraschen

put mit einem vorhandenen Schlüssel legt keinen zweiten Eintrag an, sondern überschreibt. Schlüssel sind eindeutig.

get für einen fehlenden Schlüssel liefert null. Rechnest du damit weiter, knallt es – dafür gibt es getOrDefault.

Die Reihenfolge beim Durchlaufen ist beliebig. Brauchst du die Einfügereihenfolge: LinkedHashMap. Brauchst du sortierte Schlüssel: TreeMap. Benutzung identisch, nur der Name beim Anlegen ändert sich.

Der Fall aus der Klausur: Objekte unter ihrem Namen ablegen

```
HashMap<String, Hund> hundeMap = new HashMap<>();

Hund b = new Hund("Bello", 3, "Dackel");
hundeMap.put(b.getName(), b);        // Schluessel = Name, Wert = das Objekt

hundeMap.get("Bello").getAlter()     // 3
```

Warum in der Klausur oft BEIDE vorkommen

Eine ArrayList behält die Reihenfolge, sucht aber langsam. Eine HashMap sucht schnell, hat aber keine Reihenfolge.

Verlangt die Aufgabe beides, hältst du beide Strukturen und pflegst sie in der add-Methode gemeinsam. Genau das macht die Verwalterklasse in Modul 9.

Aufgabe 5.1 Stufe 1 HashMap-Ausgaben

Voraussetzung: *5.1*

```
HashMap<String, Integer> m = new HashMap<>();
m.put("a", 1);
m.put("b", 2);
m.put("a", 9);
```

m.size() _____

m.get("a") _____

m.get("z") _____

m.getOrDefault("z", 0) _____

Aufgabe 5.2 Stufe 2 HashMap benutzen

Voraussetzung: *5.1*

Lege eine HashMap an, die zu einem Buchtitel die Seitenzahl speichert. Trage zwei Bücher ein, gib die Seitenzahl des ersten aus und prüfe, ob ein drittes enthalten ist.

Deine Lösung:

5.2 HashSet: Menge ohne Duplikate

```
HashSet<String> s = new HashSet<>();
s.add("a");
s.add("b");
s.add("a");            // nochmal a

System.out.println(s);         // [a, b]
System.out.println(s.size());  // 2   <- das zweite a wurde geschluckt
```

5.3 Das Problem mit eigenen Objekten

```
Hund h1 = new Hund("Bello", 3, "Dackel");
Hund h2 = new Hund("Bello", 3, "Dackel");   // gleiche Werte!

h1 == h2           // false
h1.equals(h2)      // false            <- auch das!

HashSet<Hund> hs = new HashSet<>();
hs.add(h1); hs.add(h2);
hs.size()          // 2                <- beide drin, obwohl gleich
```

Warum sagt Java hier „verschieden"?

Weil Java von sich aus nur eine Frage stellen kann: Liegen die beiden an derselben Stelle im Speicher? Und das tun sie nicht – new wurde zweimal aufgerufen, es sind zwei getrennte Objekte.

Dass zwei Hunde mit demselben Namen für dich derselbe Hund sind, kann Java nicht wissen. Diese Regel musst du selbst hinschreiben. Dafür gibt es equals.

5.4 equals selbst schreiben

Die Methode bekommt ein fremdes Objekt und muss true oder false sagen. Sie prüft der Reihe nach vier Dinge:

```
@Override public boolean equals(Object o) {

    if (this == o) return true;                    // 1. Bist du ich selbst?

    if (o == null || getClass() != o.getClass())   // 2. Bist du ueberhaupt
        return false;                              //    vergleichbar?

    Hund anderer = (Hund) o;                       // 3. Umwandeln (Cast)

    return name.equals(anderer.name);              // 4. Feld vergleichen
}
```

Zeile für Zeile

Zeile 1 ist eine Abkürzung. Buchstäblich dasselbe Objekt? Dann sofort fertig.

Zeile 2 fängt zwei Fälle ab: Das Objekt ist null, oder es ist gar kein Hund – jemand könnte hund.equals("Bello") mit einem String schreiben. Beides bedeutet: nicht gleich.

Zeile 3 ist der Schritt, der am meisten verwirrt. Der Parameter heißt Object, also weiß Java noch nicht, dass da ein name drinsteckt. Die Klammern davor – (Hund) o – sagen: „behandle o ab jetzt als Hund". Das heißt Cast. Erst danach kommst du an anderer.name.

Zeile 4 vergleicht das Feld, an dem die Gleichheit hängt. Weil name ein String ist: mit equals, niemals mit ==.

5.5 Und wozu hashCode?

Das Bild mit dem Schrank

Stell dir ein HashSet als Schrank mit nummerierten Fächern vor. Beim Hinzufügen berechnet es aus dem hashCode die Fachnummer und legt das Objekt dort ab.

Beim Suchen berechnet es wieder die Fachnummer und schaut nur in diesem einen Fach nach – deshalb ist es so schnell. Erst innerhalb des Fachs wird mit equals verglichen.

Jetzt der Punkt: Hätten zwei gleiche Objekte verschiedene Hashcodes, landeten sie in verschiedenen Fächern. Das Set würde sie nie vergleichen und beide behalten – obwohl equals true sagen würde.

Daher die Regel: Wer equals überschreibt, muss auch hashCode überschreiben. Und beide müssen dieselben Felder verwenden.

hashCode ist eine Zeile

```
@Override public int hashCode() {
    return Objects.hash(name);        // dasselbe Feld wie in equals
}
// bei mehreren Feldern: Objects.hash(name, alter)
```

Das Ergebnis, verifiziert

```
OHNE equals und hashCode:   h1.equals(h2) = false    HashSet size = 2
MIT  equals und hashCode:   h1.equals(h2) = true     HashSet size = 1
```

Aufgabe 5.3 Stufe 2 equals und hashCode schreiben

Voraussetzung: *5.3, 5.4 und 5.5*

Ergänze für die Klasse Buch equals und hashCode. Zwei Bücher gelten als gleich, wenn sie denselben Titel haben. Geh die vier Zeilen der Reihe nach durch.

Deine Lösung:

Aufgabe 5.4 Stufe 2 equals für Tier

Voraussetzung: *5.3 bis 5.5, Modul 3*

Ergänze für die abstrakte Klasse Tier aus Modul 3 equals und hashCode. Zwei Tiere gelten als gleich, wenn sie denselben Namen haben.

Prüfe danach gedanklich: Was gibt ein HashSet aus, in das du einen Hund „Bello" und eine Katze „Bello" legst? Denk an getClass().

Deine Lösung:

Antwort auf die Zusatzfrage: _____

Modul 6 – Interfaces und Comparable

Baut auf: Modul 3

Wird gebraucht für: Modul 7, 9, 11 – Comparable wird in Aufgabe 3 fast immer verlangt

6.1 Was ein Interface ist

Ein Interface ist eine Liste von Methoden, die eine Klasse haben muss. Es steht nur da, was sie können muss – nicht wie.

```
interface Bezahlbar {
    double preis();          // nur die Ankuendigung, kein Rumpf
}

class Katze extends Tier implements Bezahlbar {
    @Override public double preis() { return 50.0; }   // hier steht das Wie
}
```

Die zwei Punkte, die zählen

Eine Klasse kann nur EINE Klasse erweitern (extends), aber BELIEBIG VIELE Interfaces implementieren (implements).

extends kommt zuerst, dann implements. Beides zusammen ist erlaubt.

6.2 Comparable – die natürliche Ordnung

Warum das nötig ist

Java kann Zahlen und Strings sortieren, aber nicht deine Hunde – es weiß ja nicht, was „größer" bei einem Hund bedeutet. Comparable ist das Interface, mit dem du es ihm sagst.

SS 2025 war eine ganze Teilaufgabe nichts anderes als „Point2D Comparable machen". Der Grund stand in der nächsten Teilaufgabe: Dort wurde eine TreeMap verlangt, und die braucht eine Ordnung auf den Schlüsseln. Steht in einer Aufgabe TreeSet oder TreeMap, ist Comparable die versteckte Pflichtaufgabe.

```
abstract class Tier implements Comparable<Tier> {
    // ...
    @Override public int compareTo(Tier o) {
        return this.name.compareTo(o.name);      // nach Name sortieren
    }
}

ArrayList<Tier> liste = new ArrayList<>();
// ... befuellen ...
Collections.sort(liste);          // geht jetzt!
```

Was der Rückgabewert bedeutet

```
negativ  ->  this kommt VOR o
0        ->  gleichwertig
positiv  ->  this kommt NACH o
```

Du musst diese Zahl selten selbst ausrechnen. Bei Strings nimmst du compareTo, bei Zahlen Integer.compare bzw. Double.compare.

Zwei Kriterien nacheinander

```
@Override public int compareTo(Point2D o) {
    if (this.x != o.x) return Integer.compare(this.x, o.x);   // erst x
    return Integer.compare(this.y, o.y);                      // bei Gleichstand y
}
```

6.3 Comparator – beliebig viele Ordnungen

Comparable gibt genau eine Ordnung. Brauchst du mehrere, nimmst du Comparator und übergibst ihn beim Sortieren.

```
liste.sort(Comparator.comparingInt(Tier::getAlter));           // nach Alter
liste.sort(Comparator.comparingInt(Tier::getAlter).reversed()); // absteigend
liste.sort(Comparator.comparing(Tier::getName));               // nach Name
liste.sort(Comparator.naturalOrder());                         // nutzt compareTo

// mehrstufig
liste.sort(Comparator.comparingInt(Tier::getAlter)
                     .thenComparing(Tier::getName));
```

Verifiziert mit Rex(5), Anna(2), Bello(3)

```
Collections.sort(liste)                       -> Anna  Bello  Rex   (Name)
sort(comparingInt(Tier::getAlter))            -> Anna  Bello  Rex   (Alter)
sort(comparingInt(Tier::getAlter).reversed()) -> Rex   Bello  Anna
```

	Comparable<T>	Comparator<T>
Wo	in der Klasse selbst	außerhalb, als Argument
Methode	`int compareTo(T o)`	`int compare(T a, T b)`
Anzahl	genau eine	beliebig viele
Für	TreeSet, TreeMap, Collections.sort	`sort(c), sorted(c)`

Aufgabe 6.1 Stufe 1 Comparable oder Comparator

Voraussetzung: *6.2 und 6.3*

Aussage	welches von beiden?
wird in der Klasse selbst implementiert	_____
`Methode heißt compare(T a, T b)`	_____
es kann beliebig viele davon geben	_____
wird von TreeSet und TreeMap gebraucht	_____
wird als Argument an sort übergeben	_____

Aufgabe 6.2 Stufe 2 compareTo schreiben

Voraussetzung: *6.2*

Die Klasse Buch hat titel (String), jahr (int) und seiten (int). Schreibe compareTo so, dass nach dem Titel verglichen wird. Schreibe danach eine zweite Variante: erst nach Jahr, bei gleichem Jahr nach Titel.

Deine Lösung:

Aufgabe 6.3 Stufe 2 Comparator als Einzeiler

Voraussetzung: *6.3*

a) nach Seitenzahl aufsteigend: _____

b) nach Seitenzahl absteigend: _____

c) nach Jahr, dann nach Titel: _____

d) nach natürlicher Ordnung: _____

Aufgabe 6.4 Stufe 3 Point2D Comparable machen

Voraussetzung: *Modul 2, 6.2*

Die Original-Teilaufgabe aus SS 2025

Implementieren Sie eine Klasse Point2D mit den Attributen x und y (beide int). Die Klasse soll Comparable<Point2D> sein, damit sie als Schlüssel in einer TreeMap dienen kann. Verglichen wird zuerst nach x, bei gleichem x nach y.

Ergänzen Sie außerdem equals, hashCode und toString, sodass ein Punkt als "(3,4)" ausgegeben wird.

Deine Lösung:

Modul 7 – Lambda und funktionale Interfaces

Baut auf: Modul 6 (Interface-Begriff)

Wird gebraucht für: Modul 9, 11, 12 – Gewichtung +++ in der Klausur

7.1 Der ungewohnte Gedanke

Bisher hast du Zahlen und Objekte an Methoden übergeben. Jetzt übergibst du Verhalten – eine Bedingung oder eine Berechnung – als Argument.

Warum das nützlich ist: Statt zehn Methoden filtereGerade, filtereGroesserAls, filtereRot zu schreiben, schreibst du eine einzige filter und sagst ihr beim Aufruf, wonach sie filtern soll.

```
import java.util.function.*;

Predicate<Integer> gerade = n -> n % 2 == 0;

gerade.test(4)     // true
gerade.test(7)     // false
```

Den Pfeil lesen

n -> n % 2 == 0 heißt: „nimm ein n und gib zurück, ob n durch 2 teilbar ist". Links vom Pfeil der Eingabewert, rechts das Ergebnis.

Ein funktionales Interface ist ein Interface mit genau EINER abstrakten Methode. Nur deshalb funktioniert die Kurzschreibweise: Der Compiler weiß eindeutig, welche Methode gemeint ist.

7.2 Die wichtigsten funktionalen Interfaces

Interface	Methode	Nimmt	Gibt zurück
`Supplier<T>`	`T get()`	nichts	T
`Consumer<T>`	`void accept(T)`	T	nichts
`Function<T,R>`	`R apply(T)`	T	R
`Predicate<T>`	`boolean test(T)`	T	boolean
`UnaryOperator<T>`	`T apply(T)`	T	T
`BinaryOperator<T>`	`T apply(T,T)`	zweimal T	T
`BiFunction<T,U,R>`	`R apply(T,U)`	T und U	R

Beispiele für alle

```
Predicate<Integer>  gerade = n -> n % 2 == 0;         gerade.test(4)
Function<String,Integer> laenge = s -> s.length();     laenge.apply("abc")
Supplier<String>    hallo  = () -> "Hallo";           hallo.get()
Consumer<String>    druck  = s -> System.out.println(s);  druck.accept("x")
UnaryOperator<String> gross = s -> s.toUpperCase();    gross.apply("ab")
BinaryOperator<Integer> max = (a, b) -> a > b ? a : b; max.apply(3, 8)
```

7.3 Syntaxregeln

Richtig

```
x -> x * x                  // ein Parameter, Klammern optional
(x) -> x * x                // Klammern erlaubt
(x, y) -> x + y             // mehrere Parameter: Klammern PFLICHT
(int x) -> x * x            // Typangabe: Klammern PFLICHT
() -> 42                    // kein Parameter: leere Klammern PFLICHT
```

```
(a, b) -> { return a + b; }  // Block: geschweifte Klammern + return + Semikolon
```

Falsch

```
a, b -> a + b                // mehrere Parameter ohne Klammern
int a, int b -> a + b        // Typen ohne Klammern
(a, b) -> return a + b;      // return nur im Block { }
(a, b) -> (return a + b;)    // runde statt geschweifte Klammern
```

Die Regel in einem Satz

Klammern um die Parameter darfst du nur weglassen, wenn es genau ein Parameter ohne Typangabe ist. Und return gibt es nur zusammen mit geschweiften Klammern.

7.4 Wozu man das braucht

```
static ArrayList<Integer> filtere(ArrayList<Integer> liste, Predicate<Integer> p) {
    ArrayList<Integer> ergebnis = new ArrayList<>();
    for (Integer x : liste) {
        if (p.test(x)) ergebnis.add(x);       // die Bedingung anwenden
    }
    return ergebnis;
}

filtere(zahlen, gerade)          // [2, 4, 6]
filtere(zahlen, n -> n > 3)      // [4, 5, 6]
```

Das ist der Kern

Dieselbe Methode, zwei völlig verschiedene Ergebnisse – weil beim Aufruf eine andere Bedingung mitgegeben wurde. Die Methode selbst weiß gar nicht, wonach sie filtert.

Genau deshalb hat die Verwalterklasse in Modul 9 eine Methode filter(Predicate<T> p).

7.5 Methodenreferenzen

Art	Schreibweise	Beispiel
statische Methode	Klasse::methode	Integer::parseInt
Instanzmethode eines gegebenen Typs	Klasse::methode	String::length
Instanzmethode eines Objekts	objekt::methode	System.out::println
Konstruktor	Klasse::new	ArrayList::new

Wie man sie unterscheidet

Entscheidend ist, was links vom :: steht. Ein Objekt (kleingeschrieben) → Instanzmethode eines Objekts. Ein Klassenname mit statischer Methode → statisch. Ein Klassenname mit Instanzmethode → Instanzmethode eines gegebenen Typs.

Aufgabe 7.1 Stufe 1 Die Tabelle aus dem Kopf

Voraussetzung: *7.2*

Interface	Methode	Nimmt	Gibt zurück
`Supplier<T>`	_____	_____	_____
`Consumer<T>`	_____	_____	_____
`Function<T,R>`	_____	_____	_____

Interface	Methode	Nimmt	Gibt zurück
`Predicate<T>`	_____	_____	_____
`BinaryOperator<T>`	_____	_____	_____

Aufgabe 7.2 Stufe 1 Syntax: erlaubt oder nicht

Voraussetzung: *7.3*

- ☐ `(a, b) -> a + b`
- ☐ `a, b -> a + b`
- ☐ `(a, b) -> return a + b;`
- ☐ `(a, b) -> { return a + b; }`
- ☐ `x -> x * x`
- ☐ `(int a, int b) -> a + b`
- ☐ `() -> 42`

Aufgabe 7.3 Stufe 2 Lambdas schreiben

Voraussetzung: *7.2 und 7.3*

Schreibe jeweils die vollständige Zeile mit Typ, etwa: Predicate<Integer> gerade = n -> n % 2 == 0;

a) Prädikat: prüft, ob ein String länger als 5 Zeichen ist

b) Funktion: bildet einen String auf seine Länge ab

c) Funktion: bildet eine Zahl auf ihr Quadrat ab

d) Supplier: liefert den String "Hallo"

e) Consumer: gibt einen String mit vorangestelltem >> aus

f) BinaryOperator: gibt die größere zweier Zahlen zurück

g) UnaryOperator: wandelt einen String in Großbuchstaben

Aufgabe 7.4 Stufe 2 Predicate benutzen

Voraussetzung: *7.4, Modul 4*

Schreibe eine Methode ArrayList<Buch> filtere(ArrayList<Buch> liste, Predicate<Buch> p), die alle passenden Bücher zurückgibt. Rufe sie danach zweimal auf: einmal für Bücher mit mehr als 200 Seiten, einmal für Bücher, deren Titel mit F beginnt.

Deine Lösung:

Aufgabe 7.5 Stufe 2 Methodenreferenzen bestimmen

Voraussetzung: *7.5*

String::length _____

Integer::parseInt _____

ArrayList::new _____

System.out::println _____

Modul 8 – Generics

Baut auf: Modul 3 und 4

Wird gebraucht für: Modul 9 – Gewichtung +++ in der Klausur

8.1 Was <T> bedeutet

Du hast ArrayList<String> benutzt. Jetzt schreibst du selbst eine Klasse mit spitzen Klammern.

```
class Box<T> {              // T ist ein Platzhalter fuer einen Typ
    private T inhalt;
    public void setze(T x) { inhalt = x; }
    public T hole()         { return inhalt; }
}

Box<String> b1 = new Box<>();      // T wird zu String
b1.setze("Hallo");
String s = b1.hole();              // kein Cast noetig

Box<Integer> b2 = new Box<>();     // T wird zu Integer
b2.setze(42);
```

Wozu der Aufwand

Ohne Generics bräuchtest du für jeden Typ eine eigene Klasse: StringBox, IntegerBox, HundBox. Mit <T> schreibst du sie einmal.

T ist ein frei gewählter Buchstabe. Üblich sind T für Typ, E für Element, K und V für Key und Value.

8.2 Mit Schranke: T extends Tier

```
class Tierheim<T extends Tier> {
    private ArrayList<T> tiere = new ArrayList<>();

    public void add(T tier) {
        tiere.add(tier);
        System.out.println(tier.getName());   // geht, weil T ein Tier ist!
    }
}
```

Warum die Schranke nötig ist

Bei einem einfachen <T> weiß Java nichts über den Typ – du könntest keine einzige eigene Methode darauf aufrufen, nur die von Object.

T extends Tier sagt: „Was hier eingesetzt wird, ist auf jeden Fall ein Tier oder eine Unterklasse." Damit darfst du alle Methoden von Tier benutzen, also auch getName().

Genau deshalb steht in Modul 9 Tierheim<T extends Tier> und nicht nur Tierheim<T>.

8.3 Was in der Klausur gefragt wird

Eigenschaften für Multiple Choice

- Generics bieten Typprüfung zur Compilezeit und reduzieren Casting.
- Sie verwenden Type Erasure: Zur Laufzeit ist die Typinformation gelöscht.
- Primitive Typen sind nicht erlaubt – List<Integer>, nie List<int>.
- new T() und new T[] sind verboten (Folge der Type Erasure).
- Raw Types sind Generics ohne Typangabe, etwa List l = new ArrayList(). Erlaubt, aber unsicher.
- Generics sind invariant: Box<Integer> ist KEIN Box<Number>, obwohl Integer ein Number ist.

Aufgabe 8.1 Stufe 1 Erlaubt oder nicht

Voraussetzung: *8.1 und 8.3*

- ☐ `Box<Integer> b = new Box<Integer>();`
- ☐ `Box<int> b = new Box<int>();`
- ☐ `Box b = new Box();`
- ☐ `Box<String> b = new Box<>();`

Aufgabe 8.2 Stufe 2 Generische Klasse schreiben

Voraussetzung: *8.1*

Schreibe eine generische Klasse Paar<K, V> mit zwei privaten Feldern, einem Konstruktor und zwei Gettern. Lege danach ein Paar<String, Integer> an.

Deine Lösung:

Modul 9 – Die Verwalterklasse

Baut auf: Modul 3, 4, 5, 6, 7, 8 – alles bisher

Wird gebraucht für: Teil c jeder Klausuraufgabe 3

Warum dieses Modul das wichtigste ist

Aufgabe 3 der Klausur war dreimal in Folge dieselbe Aufgabe mit anderen Namen: abstrakte Oberklasse, zwei bis drei Unterklassen, eine Verwalterklasse mit einer Collection.

- SS 2024: abstract SmartDevice, Sensor und Light, Verwalter mit Map und Set
- WS 2024/25: Item, abstract Weapon, Sword, Shield, Bow
- SS 2025: abstract Shape, Circle und Rectangle, Canvas mit TreeMap

Wenn du dieses Muster auswendig kannst und nur die Namen tauschst, ist Aufgabe 3 Abschreiben statt Programmieren.

9.1 Die Bauanleitung, Schritt für Schritt

Schritt 1: Klassenkopf mit Schranke (Modul 8)

```
class Tierheim<T extends Tier> {
}
```

Schritt 2: beide Strukturen als Felder (Modul 4 und 5)

```
    private ArrayList<T> tiere = new ArrayList<>();       // fuer die Reihenfolge
    private HashMap<String, T> byName = new HashMap<>();  // fuers schnelle Suchen
```

Schritt 3: add pflegt BEIDE

```
    public void add(T tier) {
        tiere.add(tier);
        byName.put(tier.getName(), tier);
    }
```

Schritt 4: Suchen geht über die Map

```
    public T findByName(String name) {
        return byName.get(name);
    }
```

Schritt 5: getAll gibt eine KOPIE zurück

```
    public ArrayList<T> getAll() {
        return new ArrayList<>(tiere);
    }
```

Warum eine Kopie?

Gäbest du die Liste selbst zurück, könnte jemand von außen Elemente löschen, ohne dass die Map davon erfährt – dann wären die beiden Strukturen nicht mehr synchron. Mit new ArrayList<>(tiere) bekommt der Aufrufer eine eigene Liste.

Schritt 6: filter mit Predicate (Modul 7)

```
    public ArrayList<T> filter(Predicate<T> p) {
        ArrayList<T> ergebnis = new ArrayList<>();
        for (T tier : tiere) {
            if (p.test(tier)) ergebnis.add(tier);
        }
        return ergebnis;
    }
```

Schritt 7: applyToAll mit Consumer (Modul 7)

```
    public void applyToAll(Consumer<T> c) {
        for (T tier : tiere) c.accept(tier);
    }
```

Schritt 8: rechnen mit einer Schleife

```
    public double durchschnittsalter() {
        if (tiere.isEmpty()) return 0;
        int summe = 0;
        for (T tier : tiere) summe += tier.getAlter();
        return (double) summe / tiere.size();
    }
```

Schleifen oder Streams?

In Modul 11 lernst du Streams kennen – damit werden diese Methoden zu Einzeilern wie tiere.stream().filter(p).toList(). Das ist eleganter.

Die Schleifen-Variante oben ist aber genauso richtig und gibt in der Klausur volle Punkte. Schreib sie erstmal so. Umsteigen kannst du später – musst du aber nicht.

Verifizierte Ausgabe mit drei Tieren

```
getAll size    : 3
findByName Rex : 5
filter alter>2 : 2
durchschnitt   : 3.333...
```

Aufgabe 9.1 Stufe 3 Die Verwalterklasse selbst schreiben

Voraussetzung: *Modul 3 bis 8, Abschnitt 9.1*

Schreibe die Klasse Buecherregal<T extends Buch> mit einer ArrayList und einer HashMap. Implementiere add, findByTitel, getAll, filter(Predicate<T>) und gesamtSeiten(). Nimm Schleifen, keine Streams.

Deine Lösung:

Aufgabe 9.2 Stufe 3 Zwei-Ebenen-Struktur

Voraussetzung: *9.1, Modul 5*

Nach der SS-2024-Aufgabe

SS 2024 hielt der SmartDeviceManager eine Map von der Etagennummer auf ein Floor-Objekt, und jedes Floor enthielt ein Set von Geräten.

Schreibe analog eine Klasse Gebaeude mit einer HashMap<Integer, Etage>. Die Klasse Etage hält eine ArrayList<Tier>. Implementiere void addTier(int etage, Tier t), das die Etage anlegt, falls sie noch nicht existiert, und eine Methode ArrayList<Tier> alleTiere(), die über beide Ebenen läuft.

Deine Lösung:

9.2 Stufe 4 – die vollständige Klausuraufgabe

Aufgabe 9.3 Stufe 4 Fuhrparkverwaltung

Voraussetzung: *alles aus Modul 3 bis 9*

Aufgabenstellung

Ein Unternehmen möchte seinen Fuhrpark verwalten. Implementieren Sie die folgenden Teilaufgaben in Java.

a) Implementieren Sie eine abstrakte Klasse Fahrzeug mit den privaten Attributen name (String) und baujahr (int). Ergänzen Sie Konstruktor, Getter, eine abstrakte Methode double steuer() sowie toString, equals und hashCode. Zwei Fahrzeuge gelten als gleich, wenn sie denselben Namen haben. Die Klasse soll Comparable<Fahrzeug> implementieren und nach dem Namen vergleichen.

b) Implementieren Sie PKW mit dem Attribut sitze (int) und LKW mit dem Attribut tonnen (double). Die Steuer beträgt beim PKW 20 Euro je Sitz, beim LKW 30 Euro je Tonne.

c) Implementieren Sie eine generische Klasse Fuhrpark<T extends Fahrzeug>. Die Fahrzeuge sollen sowohl in Einfügereihenfolge durchlaufen als auch effizient über den Namen gefunden werden können. Begründen Sie Ihre Wahl der Datenstrukturen.

d) Implementieren Sie add, remove(String name) mit boolean-Rückgabe, findByName, getAll, filter(Predicate<T>), applyToAll(Consumer<T>), gesamtSteuer() und aeltestes().

e) Schreiben Sie eine main-Methode, die vier Fahrzeuge anlegt, alle ab Baujahr 2018 filtert und die Gesamtsteuer ausgibt.

Planen Sie etwa 35 Minuten ein – das entspricht dem Anteil dieser Aufgabe in der echten Klausur.

a) und b):

c) und d):

e) und Begründung der Datenstrukturen:

Modul 10 – Collection auswählen

Baut auf: Modul 5

Wird gebraucht für: Modul 11 – und die Begründungsfrage, die in Übung 7 und 8 mehrfach gestellt wurde

Das eigentliche Prüfungsthema

In Übung 7 und 8 stand mehrfach: „Sie müssen die geeignete Collection selbst auswählen und begründen warum Sie diese gewählt haben." Das ist keine Nebenbemerkung – genau diese Auswahlentscheidung wird geprüft.

10.1 Stufe 1 – Verstehen

Aufgabe 10.1 Stufe 1 Die Eigenschaftstabelle

Voraussetzung: *Modul 4 und 5*

Fülle aus: Erlaubt die Struktur Duplikate? Welche Reihenfolge hat sie?

Klasse	Duplikate?	Reihenfolge
ArrayList	_____	_____
HashSet	_____	_____
LinkedHashSet	_____	_____
TreeSet	_____	_____
HashMap	_____	_____
TreeMap	_____	_____
ArrayDeque	_____	_____
PriorityQueue	_____	_____

Aufgabe 10.2 Stufe 1 Ausgaben vorhersagen

Voraussetzung: *Modul 4 und 5*

Was gibt jede Zeile aus? Die Eingabe ist immer dieselbe.

new ArrayList<>(List.of(3, 1, 3)) _____

new HashSet<>(List.of(3, 1, 3)) _____

new TreeSet<>(List.of(3, 1, 3)) _____

new LinkedHashSet<>(List.of(3, 1, 3)) _____

Aufgabe 10.3 Stufe 1 Wahr oder falsch

Voraussetzung: *Modul 4 und 5*

- ☐ Map ist ein Untertyp von Collection.
- ☐ List ist ein Interface.
- ☐ ArrayList implementiert List.
- ☐ ArrayList ist threadsicher.
- ☐ Ein TreeSet braucht Comparable oder einen Comparator.
- ☐ Ein HashSet funktioniert korrekt, auch wenn hashCode nicht überschrieben ist.
- ☐ Eine Queue arbeitet standardmäßig nach dem FIFO-Prinzip.

☐ Eine PriorityQueue liefert bei poll immer das größte Element.

10.2 Stufe 2 – Einzelteile

Aufgabe 10.4 Stufe 2 Die richtige Collection wählen

Voraussetzung: *Modul 5, Abschnitt 10.1*

Trage für jede Anforderung die passende Klasse ein und begründe in einem Halbsatz.

Anforderung	Klasse	Begründung
Produkte schnell über ihre ID finden	_____	_____
keine doppelten Bücher pro Nutzer	_____	_____
Bücher sortiert nach Rückgabedatum, ohne Duplikate	_____	_____
Anfragen in Eingangsreihenfolge abarbeiten	_____	_____
Anfragen nach Priorität abarbeiten	_____	_____
in Einfügereihenfolge iterieren	_____	_____
zu jeder Quersumme alle Zeitstempel speichern	_____	_____

Aufgabe 10.5 Stufe 2 Map-Methoden

Voraussetzung: *Modul 5*

Schreibe jeweils die passende Zeile.

a) Wert zu Schlüssel "a" holen, sonst 0: _____

b) Eintrag nur setzen, wenn noch nicht vorhanden: _____

c) über alle Schlüssel-Wert-Paare iterieren (for-Schleife): _____

d) über alle Paare mit forEach und Lambda: _____

e) zu Schlüssel k eine Liste anlegen, falls sie fehlt, und x anhängen: _____

10.3 Stufe 3 – Zusammensetzen

Aufgabe 10.6 Stufe 3 Inventory nach Übung 7

Voraussetzung: *Modul 2, 5, 7*

Implementieren Sie eine Klasse Product mit den Feldern productId (int), name (String), category (String), price (double) und quantity (int), inklusive Konstruktor und Gettern.

Implementieren Sie eine Klasse Inventory, die Produkte verwaltet. Die Collection muss zwei Anforderungen erfüllen: schnelles Hinzufügen und Entfernen sowie effizientes Suchen anhand der Produkt-ID. Begründen Sie Ihre Wahl.

Implementieren Sie die Methoden addProduct, removeProduct(int productId) mit boolean-Rückgabe, findProductById, findProductsByCategory, getAllProducts, sortProductsByPrice, getLowStockProducts(int threshold), filterProducts(Predicate<Product>) und applyToProducts(Consumer<Product>).

Deine Lösung:

Aufgabe 10.7 Stufe 3 Map und Collection kombinieren

Voraussetzung: *Modul 5*

Nach Übung 7, Aufgabe 2

Ein Consumer berechnet Quersummen und soll zu jeder berechneten Quersumme alle Zeitstempel speichern, zu denen sie aufgetreten ist. Es soll außerdem effizient zählbar sein, wie oft eine bestimmte Quersumme berechnet wurde, und die Quersummen sollen aufsteigend ausgegeben werden können.

Welche Datenstruktur wählst du? Schreibe die Felddeklaration und die Methode consume(int zahl), die die Quersumme berechnet und den Zeitstempel ablegt. Ergänze numberOfOccurrences(int quersumme) und getCrossTotalsAscending().

Deine Lösung:

Modul 11 – Streams

Baut auf: Modul 7 (Lambda) und Modul 10

Wird gebraucht für: Aufgabe 4 der Klausur

Aufgabe 4 der Klausur

Das Gedankenprotokoll empfiehlt ausdrücklich die Übungsaufgaben mit den Vulkandaten oder den Schüler-Beispielen. Dieses Modul arbeitet durchgehend mit einem Studenten-Datensatz, damit du dich nicht ständig neu einlesen musst.

Grunddatensatz für alle Aufgaben dieses Moduls:

```
record Student(String name, String kurs, int note, int semester) {}

List<Student> s = List.of(
    new Student("Anna",  "Info",  2, 3),
    new Student("Ben",   "Info",  4, 1),
    new Student("Clara", "Mathe", 1, 5),
    new Student("David", "Info",  3, 3),
    new Student("Eva",   "Mathe", 2, 1),
    new Student("Felix", "BWL",   5, 7));
```

11.1 Stufe 1 – Verstehen

Aufgabe 11.1 Stufe 1 Intermediate oder terminal

Voraussetzung: *Abschnitt 11.1 und 11.2*

Trage bei jeder Operation ein I für intermediate oder ein T für terminal ein.

Operation	I/T	Operation	I/T
filter	____	collect	____
map	____	count	____
forEach	____	sorted	____
reduce	____	distinct	____
limit	____	anyMatch	____
flatMap	____	findFirst	____

Formuliere die Regel in einem Satz: _____

Aufgabe 11.2 Stufe 1 Eigenschaften

Voraussetzung: *Abschnitt 11.1*

- ☐ Ein Stream speichert die Daten selbst.
- ☐ Ein Stream verändert die Quelle.
- ☐ Ohne Terminal-Operation passiert nichts.
- ☐ Ein Stream kann mehrfach verwendet werden.
- ☐ map gibt einen Stream zurück.
- ☐ collect gibt einen Stream zurück.

11.2 Stufe 2 – Einzelteile

Aufgabe 11.3 Stufe 2 Je eine Zeile

Voraussetzung: *Modul 7, Abschnitt 11.4*

Schreibe für den Studenten-Datensatz jeweils die vollständige Stream-Zeile.

a) Anzahl aller Studenten

b) Liste aller Namen

c) Namen aller Studenten mit Note kleiner 5

d) Durchschnittsnote

e) Name des Studenten mit der besten Note

f) alle Namen, sortiert nach Note

g) Summe aller Semester

h) alle Kurse ohne Duplikate

i) alle Namen zu einem String, getrennt durch Komma

j) prüfen, ob alle bestanden haben (Note kleiner 5)

Aufgabe 11.4 Stufe 2 Ausgaben vorhersagen

Voraussetzung: *Abschnitt 11.4*

Was gibt der jeweilige Ausdruck aus?

a)

```
Stream.of(1, 2, 3).reduce(1, (x, y) -> x * y)
```

Ergebnis: _____

b)

```
Stream.of(1, 2, 3).reduce(0, (x, y) -> x + y)
```

Ergebnis: _____

c)

```
Stream.of(1, 1, 3, 4).distinct().sorted().toList()
```

Ergebnis: _____

d)

```
Arrays.asList(1, 2, 3).stream().anyMatch(x -> x > 2)
```

Ergebnis: _____

e)

```
Stream.of("a","bb","ccc").mapToInt(String::length).sum()
```

Ergebnis: _____

11.3 Stufe 3 – Zusammensetzen

Aufgabe 11.5 Stufe 3 Gruppieren

Voraussetzung: *Abschnitt 11.4*

Schreibe jeweils die vollständige Zeile.

a) Anzahl der Studenten je Kurs

b) Durchschnittsnote je Kurs

c) je Kurs der Name des besten Studenten

d) Studenten aufgeteilt in bestanden und nicht bestanden, jeweils gezählt

e) Kurse sortiert nach Anzahl der Studenten, absteigend

f) Studenten je Notenklasse: 1-2 als "gut", 3-4 als "mittel", 5 als "schlecht"

11.4 Stufe 4 – Klausurniveau

Aufgabe 11.6 Stufe 4 Bibliotheksauswertung

Voraussetzung: *Modul 6, 7 und Abschnitt 11.4*

Nach Übung 8, Aufgabe 2

Gegeben sei eine Klasse Book mit den Attributen title (String), author (String), year (int), pages (int), genre (String), rating (double) und borrowed (boolean), sowie eine List<Book> alleBuecher.

Implementieren Sie mit Java Streams eine Methode für jede der folgenden Auswertungen. Geben Sie jeweils Signatur und Rumpf an.

- die Gesamtanzahl der Seiten aller Bücher
- alle Bücher, die nach einem gegebenen Jahr erschienen sind
- alle Bücher eines gegebenen Genres
- die durchschnittliche Bewertung pro Genre
- die drei am besten bewerteten Bücher
- die Autoren mit den meisten Büchern, absteigend sortiert
- alle Bücher sortiert nach Seitenzahl
- eine gefilterte und sortierte Liste, wobei Filter und Sortierkriterium als funktionale Interfaces übergeben werden

Planen Sie etwa 25 Minuten ein.

Deine Lösung:

Modul 12 – Innere Klassen

Baut auf: Modul 2 und 7
Wird gebraucht für: Gewichtung + in der Klausur, kam WS 2024/25 als Fehlersuchaufgabe

12.1 Stufe 1 – Verstehen

Aufgabe 12.1 Stufe 1 Die vier Arten

Voraussetzung: *Abschnitt 12.1*

Art	Wo definiert	Zugriff auf	Besonderheit
statische innere Klasse	_____	_____	_____
echte innere Klasse	_____	_____	_____
lokale Klasse	_____	_____	_____
anonyme Klasse	_____	_____	_____

Aufgabe 12.2 Stufe 1 Instanziierung

Voraussetzung: *Abschnitt 12.1*

Wie erzeugt man ein Objekt einer statischen inneren Klasse? _____

Wie erzeugt man ein Objekt einer echten inneren Klasse? _____

Aufgabe 12.3 Stufe 1 Shadowing – Ausgabe vorhersagen

Voraussetzung: *Abschnitt 12.1*

```
public class Outer {
    public static int i = 0;
    public void test() {
        class Inner {
            private int i = 1;
            public void print(int i) {
                System.out.println("i: " + i);
                System.out.println("i: " + this.i);
                System.out.println("i: " + Outer.i);
            }
        }
        new Inner().print(2);
    }
}
```

Ausgabe:

12.2 Stufe 2 und 3

Aufgabe 12.4 Stufe 2 Anonyme Klasse schreiben

Voraussetzung: *Modul 6 und 7*

Gegeben ist das Interface BinaryOperation mit der Methode int apply(int a, int b). Schreibe eine Klasse Calculator mit der Methode int compute(int a, int b, BinaryOperation op) und rufe sie viermal auf – einmal je Grundrechenart, jeweils mit einer anonymen Klasse.

Deine Lösung:

Aufgabe 12.5 Stufe 3 Iterator als innere Klasse

Voraussetzung: *Modul 4 und Abschnitt 12.1*

Nach Übung 3, Aufgabe 2

Eine Klasse Lager verwaltet ein Array von Fach-Objekten, jedes mit einem Typ. Implementieren Sie:

- ein inneres Interface Iterator mit boolean hasNext() und Fach next(), wobei next eine NoSuchElementException wirft, wenn nichts mehr da ist
- eine private innere Klasse LagerIterator, die dieses Interface implementiert und über den Konstruktor einen Typ gesetzt bekommt – sie iteriert nur über Fächer dieses Typs
- eine public-Methode, die ein Iterator-Objekt zurückgibt

Erklären Sie außerdem in zwei Sätzen, welchen Vorteil die Implementierung als innere Klasse hat.

Deine Lösung:

Modul 13 – Rekursion

Baut auf: Modul 1 und 2
Wird gebraucht für: Gewichtung +, kommt fast sicher als Wissensfrage und Code-Ausgabe

13.1 Stufe 1 – Verstehen

Aufgabe 13.1 Stufe 1 Die Standardfragen

Voraussetzung: *Abschnitt 13.1*

Wann endet eine Rekursion? _____

Wo werden die rekursiven Aufrufe gespeichert? _____

Was passiert bei unendlicher Rekursion? _____

Warum ist Rekursion oft langsamer als Iteration? _____

Aufgabe 13.2 Stufe 1 Der Bauplan

Voraussetzung: *Abschnitt 13.1*

Ergänze die drei Bestandteile jeder rekursiven Methode.

```
public static Typ methode(Parameter...) {
    if (_________________)  return _________________;   // 1.
    // 2. ___________________________________________
    return ____________________________________________;  // 3.
}
```

13.2 Stufe 2 – Einzelteile

Aufgabe 13.3 Stufe 2 Kleine rekursive Methoden

Voraussetzung: *Modul 1, Abschnitt 13.2*

Schreibe jeweils eine rekursive Methode. Keine Schleifen.

a) int summe(int n) – Summe aller Zahlen von 1 bis n

b) int anzahlZiffern(int n) – wie viele Ziffern hat n?

c) int quersumme(int n) – Summe der Ziffern

d) int potenz(int basis, int exponent)

e) int ggT(int a, int b) – nach Euklid

f) boolean istPalindrom(String s, int l, int r)

g) int maxArray(int[] a, int i) – Maximum ab Index i

13.3 Stufe 3 – Zusammensetzen

Aufgabe 13.4 Stufe 3 Türme von Hanoi

Voraussetzung: *Abschnitt 13.2*

Nach Übung 2, Aufgabe 4

Implementieren Sie eine rekursive Lösung für die Türme von Hanoi. Die Methode soll bei jedem Zug ausgeben, von welchem Stab zu welchem eine Scheibe bewegt wird.

Erklären Sie danach in drei bis vier Sätzen, warum der Algorithmus das Problem löst – so, wie es im Review verlangt wurde. Geben Sie außerdem an, wie viele Züge für n Scheiben nötig sind.

Deine Lösung:

Aufgabe 13.5 Stufe 3 Iterativ und rekursiv

Voraussetzung: *Modul 6, Abschnitt 13.2*

Nach Übung 2, Aufgabe 3

Definieren Sie ein Interface Palindrom mit der Methode boolean istPalindrom(String wort). Implementieren Sie zwei Klassen PalindromRekursiv und PalindromIterativ.

Erklären Sie außerdem, warum die rekursive Variante bei wachsender Wortlänge langsamer wird als die iterative.

Deine Lösung:

Modul 14 – C: Grundlagen und Pointer

Baut auf: Modul 1 (Arrays und Schleifen) – Java-Kenntnisse werden NICHT gebraucht

Wird gebraucht für: Modul 15 – Gewichtung ++ in der Klausur

Gewichtung ++ und in jeder bekannten Klausur

SS 2024 rekursives Minimum mit Pointer-Ausgabeparameter, WS 2024/25 isPalindrome mit drei Argumenten und Pointer-Tauschen, SS 2025 Adressfragen und Pointer-Arithmetik. Die Form ist immer dieselbe: eine Funktion, die über einen Pointer ein Ergebnis nach außen schreibt.

14.1 Stufe 1 – Verstehen

Aufgabe 14.1 Stufe 1 Die zwei Operatoren

Voraussetzung: *Abschnitt 14.1*

& heißt _____

und liefert _____

* heißt _____

und liefert _____

&*p ist dasselbe wie _____

Aufgabe 14.2 Stufe 1 Werte bestimmen

Voraussetzung: *Abschnitt 14.2*

```
int  a[5] = {10, 20, 30, 40, 50};
int* p    = a;
```

Trage die Werte ein.

Ausdruck	Wert	Ausdruck	Wert
*p	_____	p[3]	_____
*(p + 3)	_____	*(p) + 3	_____
p[0]	_____	*(p + 4)	_____
nach p += 2 dann *p	_____	a == &a[0]	_____

Erkläre den Unterschied zwischen *(p+3) und *(p)+3: _____

Aufgabe 14.3 Stufe 1 Die Klausurfrage SS 2025

Voraussetzung: *Abschnitt 14.1*

```
long  l  = 140732807899964;
long* pl = &l;
printf("%p\n", &*pl);
printf("%p\n", &pl);
printf("%p\n", *pl);

Ausgabe:  0x7ffee9ca2b38
          0x7ffee9ca2b30
          0x7ffee905eb3c
```

Welche der drei Ausgaben ist die Adresse von l?

Antwort: _____

Kreuze alle zutreffenden Aussagen an.

- ☐ &*pl und pl liefern die gleiche Adresse.
- ☐ &*pl liefert die Adresse von l.
- ☐ &pl liefert die Adresse des Zeigers selbst.
- ☐ *pl dereferenziert den Zeiger und gibt den Wert von l.
- ☐ Alle drei Ausgaben sind identisch.

14.2 Stufe 2 – Einzelteile

Aufgabe 14.4 Stufe 2 Funktionen mit Pointer-Parametern

Voraussetzung: *Abschnitt 14.1 und 14.2*

a) void verdopple(int x) – verdoppelt den Wert, auf den x zeigt*

b) void tausche(int a, int* b) – vertauscht zwei Werte*

c) int summeArray(int a, int n) – Summe eines Arrays, iterativ*

d) void minMax(int a, int n, int* mn, int* mx) – gibt Minimum UND Maximum über Pointer zurück*

Aufgabe 14.5 Stufe 2 Call by value nachweisen

Voraussetzung: *Abschnitt 14.1*

```
void erhoehe(int x)   { x = x + 1; }
void erhoehe2(int* x) { *x = *x + 1; }
```

```
int a = 5;
erhoehe(a);    printf("%d\n", a);
erhoehe2(&a);  printf("%d\n", a);
```

Ausgabe und Begründung:

14.3 Stufe 3 – Zusammensetzen

Aufgabe 14.6 Stufe 3 Rekursiv mit Pointer-Ausgabeparameter

Voraussetzung: *Modul 13 (Rekursion), Abschnitt 14.3*

Das ist das Kernrezept des C-Teils. Schreibe jede Funktion rekursiv, ohne Schleife.

a) void summeRek(int a, int n, int i, int* s) – summiert das Array auf*

b) void maxRek(int a, int n, int i, int* m) – findet das Maximum*

c) void zaehleGerade(int a, int n, int i, int* c) – zählt gerade Zahlen*

Der Bauplan, der bei allen dreien gleich ist

```
1. if (index >= size) return;          Basisfall
2. Bedingung pruefen
3. *pointer = ...   ueber Pointer schreiben
4. funktion(..., index + 1, pointer);  weiter
```

Aufgabe 14.7 Stufe 3 Palindrom in C

Voraussetzung: *Abschnitt 14.3*

Die WS-2024/25-Aufgabe

Implementieren Sie int isPalindrome(const char* s, int left, int right) rekursiv. Schreiben Sie danach eine iterative Variante int isPalindromeIter(const char* s), die die Grenzen selbst bestimmt.

Deine Lösung:

14.4 Stufe 4 – Klausurniveau

Aufgabe 14.8 Stufe 4 Vollständige C-Aufgabe

Voraussetzung: *Abschnitt 14.3, Modul 13*

Aufgabenstellung

a) Implementieren Sie eine rekursive Funktion void findMin(int* input, int size, int index, int* minNumber), die das Minimum eines Arrays bestimmt und über den Pointer minNumber zurückgibt.

b) Schreiben Sie ein Hauptprogramm, das ein Array mit sechs Werten anlegt, die Funktion korrekt aufruft und das Ergebnis ausgibt. Achten Sie darauf, womit minNumber vor dem Aufruf initialisiert werden muss und warum.

c) Erklären Sie in zwei bis drei Sätzen, warum das Ergebnis über einen Pointer zurückgegeben werden muss und nicht einfach über return zurückgegeben werden kann.

d) Erweitern Sie die Funktion so, dass sie zusätzlich über einen zweiten Pointer den Index des Minimums zurückgibt.

Deine Lösung:

Modul 15 – C: Structs, Arrays und Matrizen

Baut auf: Modul 14
Wird gebraucht für: Gewichtung ++ – Übung 11 war komplett diesem Thema gewidmet

15.1 Stufe 1 – Verstehen

Aufgabe 15.1 Stufe 1 Grundlagen

Voraussetzung: *Modul 14*

Mit welchem Schlüsselwort definiert man eine Struktur? _____

Wozu dient typedef zusätzlich? _____

Wann Punkt, wann Pfeil beim Zugriff? _____

Was ist der Unterschied zwischen struct und union? _____

Warum übergibt man große structs als Pointer? _____

Aufgabe 15.2 Stufe 1 Die Matrix-Formel

Voraussetzung: *Abschnitt 15.1*

Ein zweidimensionales Array liegt im Speicher zeilenweise hintereinander. Ergänze die Umrechnung.

```
matrix[i][j]    entspricht    m[ ____________________ ]
```

Warum haben alle Übungsfunktionen die Signatur (int* m, int rows, int cols)? _____

15.2 Stufe 2 – Einzelteile

Aufgabe 15.3 Stufe 2 Struct definieren und nutzen

Voraussetzung: *Abschnitt 15.1*

Definiere ein struct Student mit name (String, max. 20 Zeichen), alter (int) und note (float). Schreibe dann:

- void printStudent(const Student* s) – gibt die Daten formatiert aus
- float schnitt(Student* arr, int n) – berechnet die Durchschnittsnote
- int findeIndex(Student* arr, int n, const char* name) – liefert den Index oder -1

Deine Lösung:

Aufgabe 15.4 Stufe 2 Matrixfunktionen

Voraussetzung: *Abschnitt 15.1*

a) int summeZeile(int m, int rows, int cols, int row)*

b) int maxSpalte(int m, int rows, int cols, int col)*

c) void swapRows(int m, int rows, int cols, int r1, int r2)*

d) void transpose(int m, int n) – nur für quadratische Matrizen*

15.3 Stufe 3 – Klausurniveau

Aufgabe 15.5 Stufe 3 Mitarbeiterdatenbank

Voraussetzung: *Abschnitt 15.1 und 15.2*

Nach Übung 11, Aufgabe 1

Entwickeln Sie ein Programm in C, das eine Datenbank für Mitarbeiterinformationen mithilfe von Arrays und Structs verwaltet.

- Definieren Sie ein struct Mitarbeiter mit Name, Alter, Abteilung und Gehalt.
- Legen Sie ein Array für bis zu 50 Mitarbeiter an.
- Implementieren Sie addMitarbeiter, deleteMitarbeiter(char* name), queryMitarbeiter(char* name) und printAllMitarbeiter.
- Ergänzen Sie Fehlerbehandlung für ein volles Array und für einen nicht gefundenen Mitarbeiter.

Achten Sie beim Löschen darauf, dass kein Loch im Array entsteht.

Deine Lösung:

Modul 16 – Was gibt der Code aus?

Baut auf: Modul 1 bis 15

Wird gebraucht für: Aufgabe 2 der Klausur – die schnellsten Punkte

Aufgabe 2 der Klausur

Diese Aufgaben sind die schnellsten Punkte der ganzen Klausur, wenn du die Muster kennst. Schreibe die Ausgabe hin, ohne sie im Kopf zu überspringen – gerade bei Rekursion mit Konsolenausgabe verrechnet man sich sonst.

Aufgabe 16.1 Stufe 2 Ausgabe bestimmen

```
public static int fac(int n) {
    if (n <= 1) { System.out.println("B"); return 1; }
    else return n * fac(n - 1);
}
// main: System.out.println(fac(3));
```

Ausgabe:

Aufgabe 16.2 Stufe 2 Ausgabe bestimmen

```
public static void zaehle(int n) {
    if (n == 0) return;
    System.out.println("vor " + n);
    zaehle(n - 1);
    System.out.println("nach " + n);
}
// main: zaehle(3);
```

Ausgabe:

Aufgabe 16.3 Stufe 2 Ausgabe bestimmen

```
int result = Stream.of(1, 2, 3, 4).reduce(2, (x, y) -> x * y);
System.out.println(result);
```

Ausgabe:

Aufgabe 16.4 Stufe 2 Ausgabe bestimmen

```
List<String> l = List.of("Anna", "Ben", "Clara");
System.out.println(l.stream().filter(s -> s.length() > 3)
                    .map(String::toUpperCase).toList());
```

Ausgabe:

Aufgabe 16.5 Stufe 2 Ausgabe bestimmen

```
Set<Integer> a = new HashSet<>(List.of(5, 3, 5, 1));
Set<Integer> b = new TreeSet<>(List.of(5, 3, 5, 1));
Set<Integer> c = new LinkedHashSet<>(List.of(5, 3, 5, 1));
System.out.println(a.size() + " " + b + " " + c);
```

Ausgabe:

Aufgabe 16.6 Stufe 2 Ausgabe bestimmen

```
Function<Integer,Integer> f = x -> x + 1;
Function<Integer,Integer> g = x -> x * 10;
System.out.println(f.andThen(g).apply(3));
System.out.println(f.compose(g).apply(3));
```

Ausgabe:

Aufgabe 16.7 Stufe 2 Ausgabe bestimmen

```
Map<String,Integer> m = new TreeMap<>();
m.put("b", 2); m.put("a", 1); m.put("b", 5);
System.out.println(m);
System.out.println(m.getOrDefault("z", 0));
```

Ausgabe:

Aufgabe 16.8 Stufe 2 Ausgabe bestimmen

```
int a[5] = {2, 4, 6, 8, 10};
int* p = a;
p++;
printf("%d %d %d\n", *p, *(p + 2), *(p) + 2);
```

Ausgabe:

Aufgabe 16.9 Stufe 2 Ausgabe bestimmen

```
int x = 7, y = 2;
printf("%d\n", x / y);
printf("%.2f\n", (double) x / y);
printf("%d\n", x % y);
```

Ausgabe:

Aufgabe 16.10 Stufe 2 Ausgabe bestimmen

```
Deque<Integer> d = new ArrayDeque<>();
d.offer(1); d.offer(2); d.offer(3);
System.out.println(d.poll());
Queue<Integer> q = new PriorityQueue<>(List.of(5, 1, 3));
System.out.println(q.poll());
```

Ausgabe:

Aufgabe 16.11 Stufe 2 Ausgabe bestimmen

```
typedef union { int i; char c[4]; } U;
U u; u.i = 66;
printf("%zu %d %c\n", sizeof(U), u.i, u.c[0]);
```

Ausgabe:

Aufgabe 16.12 Stufe 2 Ausgabe bestimmen

```
List<Integer> l = List.of(1, 2, 3, 4, 5, 6);
System.out.println(l.stream()
    .collect(Collectors.partitioningBy(x -> x % 2 == 0,
                                       Collectors.counting())));
```

Ausgabe:

Modul 17 – Probeklausur

Baut auf: alle Module

Wird gebraucht für: die Generalprobe

So gehst du vor

Setz dich 120 Minuten hin, ohne Unterlagen, ohne Rechner, ohne Handy. Schreib alles von Hand. Erst danach in Teil Z nachsehen.

Der Aufbau entspricht der echten Klausur: Multiple Choice, kurze Wissensfragen, eine Java-Programmieraufgabe nach UML, dann Streams und C.

Denk an die wichtigste Regel: Im Multiple-Choice-Teil gibt es keine Minuspunkte. Lass kein Kästchen leer.

Aufgabe 1 – Multiple Choice

Kreuze alle zutreffenden Antworten an. Es können eine, mehrere oder alle richtig sein.

1.1 Wann kommt eine Rekursion zu Ende?

- ☐ Wenn der Basisfall erreicht ist.
- ☐ Wenn der Stack überläuft.
- ☐ Wenn der Rückgabewert null ist.
- ☐ Wenn die Bedingung der Rekursion false wird.

1.2 Wo werden die rekursiven Aufrufe gespeichert?

- ☐ Im Heap.
- ☐ Im Stack.
- ☐ In globalen Variablen.
- ☐ Im Datenbereich des Programms.

1.3 Welche Eigenschaften haben innere Klassen?

- ☐ Sie haben Zugriff auf private Felder der äußeren Klasse.
- ☐ Sie können static oder non-static sein.
- ☐ Sie müssen in einer eigenen Datei stehen.
- ☐ Sie dürfen nicht public sein.

1.4 Auf welche Variablen kann aus einer lokalen Klasse zugegriffen werden?

- ☐ Auf lokale Variablen der Methode, wenn sie effektiv final sind.
- ☐ Auf Parameter der Methode, wenn sie effektiv final sind.
- ☐ Auf statische Variablen anderer Klassen.
- ☐ Auf lokale Variablen anderer Methoden.

1.5 Welche Eigenschaften haben anonyme Klassen?

- ☐ Sie haben keinen Klassennamen.
- ☐ Sie werden direkt bei der Objekterzeugung geschrieben.
- ☐ Sie können nur einmal instanziiert werden.
- ☐ Sie können mehrere Interfaces gleichzeitig implementieren.

1.6 Welche Aussagen zu Generics treffen zu?

- ☐ Sie bieten Typprüfung zur Compilezeit.
- ☐ Sie unterstützen Type Erasure.
- ☐ Sie können primitive Typen wie int direkt verwenden.

- ☐ Sie reduzieren Casting zur Laufzeit.
- ☐ Sie können Wildcards als Platzhalter haben.

1.7 Welche Objekterzeugung ist erlaubt?

- ☐ Box<Integer> b = new Box<Integer>();
- ☐ Box<int> b = new Box<int>();
- ☐ Box b = new Box();
- ☐ Box<String> b = new Box<>();

1.8 Welche Lambda-Ausdrücke sind syntaktisch korrekt?

- ☐ (a, b) -> a + b
- ☐ a, b -> a + b
- ☐ (a, b) -> { return a + b; }
- ☐ (int a, int b) -> a + b
- ☐ (a, b) -> return a + b;

1.9 Welches funktionale Interface nimmt ein Argument und gibt kein Ergebnis zurück?

- ☐ Consumer<T>
- ☐ Supplier<T>
- ☐ Function<T,R>
- ☐ Predicate<T>

1.10 Welche Aussagen treffen auf List zu?

- ☐ List ist ein Interface.
- ☐ List enthält nur eindeutige Elemente.
- ☐ List erlaubt Duplikate.
- ☐ List hat eine feste Länge.
- ☐ List erhält Elemente in Einfügereihenfolge.

1.11 Welche der folgenden sind Terminal-Operationen?

- ☐ forEach()
- ☐ map()
- ☐ reduce()
- ☐ collect()
- ☐ filter()

1.12 Welche Eigenschaften hat C?

- ☐ Prozedurale Sprache.
- ☐ Unterstützt Klassen und Objekte.
- ☐ Kompiliert direkt zu Maschinencode.
- ☐ Hat Garbage Collection.
- ☐ Unterstützt Zeigerarithmetik.

1.13 Was ist die Aufgabe eines Linkers?

- ☐ Verknüpft Objektdateien zu einem Programm.
- ☐ Übersetzt Quellcode zu Bytecode.
- ☐ Lädt das Programm in den Speicher.
- ☐ Optimiert den Code zur Compilezeit.

1.14 Mit welchem Schlüsselwort definiert man eine Struktur in C?

- ☐ class
- ☐ typedef
- ☐ struct
- ☐ record

1.15 Welche Aussagen zu TreeMap treffen zu?

- ☐ Die Schlüssel sind sortiert.
- ☐ Die Schlüssel müssen Comparable sein oder ein Comparator muss übergeben werden.
- ☐ TreeMap ist ein Untertyp von Collection.
- ☐ TreeMap erlaubt doppelte Schlüssel.

Aufgabe 2 – Kurze Wissensfragen

Antworten Sie in ganzen Sätzen.

a) Was ist ein funktionales Interface?

b) Was passiert, wenn eine Rekursion unendlich weiterläuft?

c) Warum muss hashCode überschrieben werden, wenn equals überschrieben wird?

d) Welcher Operator liefert in C die Adresse einer Variablen, welcher dereferenziert?

e) Was bedeutet Type Erasure und welche Folge hat es?

Was geben die folgenden Ausschnitte aus?

f)

```
int r = Stream.of(1, 2, 3).reduce(1, (x, y) -> x * y);
System.out.println(r);
```

Ausgabe:

g)

```
int a[5] = {10, 20, 30, 40, 50};
```

```
int* p = a;
printf("%d\n", *(p + 2));
```

Ausgabe:

h)

```
System.out.println(new TreeSet<>(List.of(4, 2, 4, 1)));
```

Ausgabe:

Aufgabe 3 – Java-Programmierung

Musikbibliothek

Gegeben ist folgendes Diagramm in Textform:

```
abstract Medium  implements Comparable<Medium>
  - titel    : String
  - laenge   : int          (in Sekunden)
  + Medium(titel, laenge)
  + getTitel() : String
  + getLaenge(): int
  + gebuehr()  : double     ABSTRAKT
  + toString() : String

Song extends Medium
  - kuenstler : String
  + gebuehr() -> laenge * 0.01

Hoerbuch extends Medium
  - autor : String
  + gebuehr() -> laenge * 0.005

Bibliothek<T extends Medium>
  + add(T) : void
  + findByTitel(String) : T
  + getAll() : List<T>
  + filter(Predicate<T>) : List<T>
  + sortedBy(Comparator<T>) : List<T>
  + gesamtGebuehr() : double
  + gruppiereNachTyp() : Map<String, List<T>>
```

a) Implementieren Sie Medium vollständig, inklusive equals und hashCode (gleich bei gleichem Titel) und compareTo (Vergleich nach Titel).

b) Implementieren Sie Song und Hoerbuch.

c) Implementieren Sie Bibliothek. Die Medien sollen sowohl in Einfügereihenfolge durchlaufen als auch schnell über den Titel gefunden werden können. Begründen Sie Ihre Wahl der Datenstrukturen.

d) Schreiben Sie eine main-Methode, die vier Medien anlegt, alle mit einer Länge über 200 Sekunden filtert und die Gesamtgebühr ausgibt.

a) und b):

c) und d):

Aufgabe 4 – Streams und C

Teil 1: Streams

Gegeben sei List<Song> songs mit den Attributen titel, kuenstler, genre, laenge (int, Sekunden), bewertung (double) und abrufe (int).

Schreiben Sie für jede Auswertung eine Stream-Zeile oder eine kurze Methode:

- die Anzahl aller Songs
- die durchschnittliche Länge
- alle Titel des Genres "Rock"
- die Gesamtlänge aller Songs eines Genres in Minuten
- die Anzahl der Songs je Künstler
- die durchschnittliche Bewertung je Künstler
- die drei meistabgerufenen Songs
- je Genre den längsten Song
- alle Künstler mit mindestens einem Song über Bewertung 4.5
- die Genres sortiert nach Anzahl der Songs, absteigend

Deine Lösung:

Teil 2: C

a) Implementieren Sie eine rekursive Funktion void zaehleGroesser(int* a, int n, int i, int grenze, int* anzahl), die zählt, wie viele Elemente des Arrays größer als grenze sind, und das Ergebnis über den Pointer anzahl zurückgibt.

b) Schreiben Sie ein Hauptprogramm, das die Funktion korrekt aufruft.

c) Definieren Sie ein struct Punkt mit den Feldern x und y (beide int) und schreiben Sie eine Funktion, die den Abstand zweier Punkte als double zurückgibt und zusätzlich den Differenzvektor über einen Pointer ausgibt.

Deine Lösung:

Teil Z – Lösungen

Wie du damit umgehst

Vergleiche nicht nur, ob es „ungefähr stimmt". Achte auf die Details: super als erste Zeile, @Override, die richtigen Klammern beim Lambda, der Stern beim Pointer. Genau daran hängen die Punkte.

Wenn eine Aufgabe nicht saß: Lösung zuklappen, Aufgabe am nächsten Tag komplett neu schreiben. Nicht abschreiben – das bringt nichts.

Die Lösungen sind nach Aufgabennummer sortiert, also in derselben Reihenfolge wie die Module.

Lösungen Modul 1 – Variablen, Arrays, Schleifen

Lösung 1.1

```
int[] a = {4, 8, 15};
for (int x : a) {
    System.out.println(x);
}
```

Lösung 1.2

verifiziert

```
static int summe(int[] a) {
    int s = 0;
    for (int x : a) s += x;
    return s;
}

static int maximum(int[] a) {
    int max = a[0];                    // mit dem ERSTEN Element starten,
    for (int i = 1; i < a.length; i++) //  nicht mit 0!
        if (a[i] > max) max = a[i];
    return max;
}
```

Bei maximum ist der Startwert entscheidend: Beginnst du mit 0 und alle Werte sind negativ, kommt fälschlich 0 heraus. Deshalb immer mit dem ersten Element starten.

Lösungen Modul 2 – Klassen und Objekte

Lösung 2.1

Feld = B, Konstruktor = D, Getter = A, toString = C, private = E

Lösung 2.2

```
class Buch {
    private String titel;
    private int seiten;

    public Buch(String titel, int seiten) {
        this.titel  = titel;
        this.seiten = seiten;
    }
    public String getTitel()  { return titel; }
    public int    getSeiten() { return seiten; }

    @Override public String toString() {
        return titel + " (" + seiten + " Seiten)";
    }
}
```

Lösung 2.3

a) Der Konstruktor hat ein void. Ein Konstruktor hat KEINEN Rückgabetyp – dadurch wird er hier zu einer ganz normalen Methode, die zufällig Auto heißt.

b) marke = marke weist den Parameter sich selbst zu. Das Feld bleibt leer. Richtig ist this.marke = marke.

c) marke ist private, von außen kommt man nicht heran. Es braucht einen Getter: a.getMarke().

Lösungen Modul 3 – Vererbung

Lösung 3.1

Richtig: Eine abstrakte Klasse kann einen Konstruktor haben. super(…) muss die erste Anweisung sein. Jede abstrakte Methode muss in jeder konkreten Unterklasse implementiert werden.

Falsch: Von einer abstrakten Klasse kann man keine Objekte erzeugen. Auf private Felder der Oberklasse kommt man nur über Getter. Eine Klasse kann nur EINE Klasse erweitern. Eine abstrakte Methode hat KEINEN Rumpf.

Lösung 3.2

```
abstract class Fahrzeug {
    private String name;

    public Fahrzeug(String name) {
        this.name = name;
    }
    public abstract double steuer();
}

class PKW extends Fahrzeug {
    private int sitze;
    public PKW(String name, int sitze) {
        super(name);
        this.sitze = sitze;
    }
    @Override
    public double steuer() { return sitze * 20.0; }
}
```

Lösung 3.3

```
abstract class Tier {
    private final String name;
    private final int alter;

    public Tier(String name, int alter) {
        this.name  = name;
        this.alter = alter;
    }
    public String getName()  { return name; }
    public int    getAlter() { return alter; }

    public abstract String laut();

    @Override public String toString() {
        return name + " (" + alter + ")";
    }
}
```

Lösung 3.4

verifiziert

```
class Hund extends Tier {
    private final String rasse;
```

```
    public Hund(String name, int alter, String rasse) {
        super(name, alter);
        this.rasse = rasse;
    }
    public String getRasse() { return rasse; }
    @Override public String laut() { return "Wau"; }
}

class Katze extends Tier {
    private final boolean stubentiger;
    public Katze(String name, int alter, boolean stubentiger) {
        super(name, alter);
        this.stubentiger = stubentiger;
    }
    public boolean isStubentiger() { return stubentiger; }
    @Override public String laut() { return "Miau"; }
}
```

Lösung 3.5

a) Eine abstrakte Methode darf keinen Rumpf haben. Richtig: public abstract double flaeche();

b) super(…) muss die ERSTE Anweisung im Konstruktor sein, hier steht es nach this.radius = radius.

c) name ist private in Tier, die Unterklasse kommt nicht direkt heran. Es braucht getName().

d) Fisch implementiert die abstrakte Methode laut() nicht. Entweder implementieren oder Fisch selbst abstract deklarieren.

Lösung 3.6

```
abstract class Konto {
    private final String inhaber;
    private double stand;

    public Konto(String inhaber, double stand) {
        this.inhaber = inhaber;
        this.stand   = stand;
    }
    public String getInhaber() { return inhaber; }
    public double getStand()   { return stand; }
    public void einzahlen(double betrag) { stand += betrag; }

    public abstract double zinsen();

    @Override public String toString() { return inhaber + ": " + stand; }
}

class Girokonto extends Konto {
    private final double dispo;
    public Girokonto(String inhaber, double stand, double dispo) {
        super(inhaber, stand);
        this.dispo = dispo;
    }
    public double getDispo() { return dispo; }
    @Override public double zinsen() { return 0.0; }
}

class Sparkonto extends Konto {
    private final double zinssatz;
    public Sparkonto(String inhaber, double stand, double zinssatz) {
        super(inhaber, stand);
        this.zinssatz = zinssatz;
    }
    @Override public double zinsen() { return getStand() * zinssatz; }
}
```

Beachte: zinsen() in Sparkonto muss getStand() benutzen, nicht stand – das Feld ist private in der Oberklasse.

Lösungen Modul 4 – ArrayList

Lösung 4.1

verifiziert

```
l.size()                  3
l.get(0)                  a
l.contains("b")           true
l nach l.remove("a")      [b, a]
```

remove entfernt nur das ERSTE passende Element, nicht alle. Deshalb bleibt das zweite a übrig.

Lösung 4.2

```
ArrayList<Buch> buecher = new ArrayList<>();
buecher.add(new Buch("Faust", 280));
buecher.add(new Buch("Werther", 150));

for (Buch b : buecher) {
    System.out.println(b.getTitel());
}
System.out.println(buecher.size());
```

Lösung 4.3

```
static Buch findeNachTitel(ArrayList<Buch> liste, String titel) {
    for (Buch b : liste) {
        if (b.getTitel().equals(titel)) return b;
    }
    return null;                    // nichts gefunden
}
```

Genau diese Schleife macht eine HashMap überflüssig – deshalb kommt sie in Modul 5. Bei 500 Elementen läuft sie im schlimmsten Fall 500 Mal, die Map findet sofort.

Lösungen Modul 5 – HashMap, HashSet, equals

Lösung 5.1

verifiziert

```
m.size()                  2      <- das zweite put auf "a" ueberschreibt nur
m.get("a")                9
m.get("z")                null
m.getOrDefault("z", 0)    0
```

Lösung 5.2

```
HashMap<String, Integer> seiten = new HashMap<>();
seiten.put("Faust", 280);
seiten.put("Werther", 150);

System.out.println(seiten.get("Faust"));            // 280
System.out.println(seiten.containsKey("Odyssee"));  // false
```

Lösung 5.3

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Buch anderes = (Buch) o;
```

```
    return titel.equals(anderes.titel);
}

@Override public int hashCode() {
    return Objects.hash(titel);
}
```

Lösung 5.4

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Tier anderes = (Tier) o;
    return name.equals(anderes.name);
}

@Override public int hashCode() {
    return Objects.hash(name);
}
```

Antwort auf die Zusatzfrage

Ein Hund „Bello" und eine Katze „Bello" gelten als VERSCHIEDEN, das HashSet enthält beide. Grund: getClass() != o.getClass() schlägt zu – Hund und Katze sind unterschiedliche Klassen, auch wenn beide von Tier erben.

Zwei Hunde mit dem Namen „Bello" gelten dagegen als gleich, das Set behält nur einen. Verifiziert: HashSet size = 1.

Lösungen Modul 6 – Interfaces und Comparable

Lösung 6.1

in der Klasse selbst = Comparable · compare(T a, T b) = Comparator · beliebig viele = Comparator · von TreeSet/TreeMap gebraucht = Comparable · als Argument an sort = Comparator

Lösung 6.2

```
// nach Titel
@Override public int compareTo(Buch o) {
    return this.titel.compareTo(o.titel);
}

// erst Jahr, dann Titel
@Override public int compareTo(Buch o) {
    if (this.jahr != o.jahr) return Integer.compare(this.jahr, o.jahr);
    return this.titel.compareTo(o.titel);
}
```

Lösung 6.3

```
a) Comparator.comparingInt(Buch::getSeiten)
b) Comparator.comparingInt(Buch::getSeiten).reversed()
c) Comparator.comparingInt(Buch::getJahr).thenComparing(Buch::getTitel)
d) Comparator.naturalOrder()
```

Lösung 6.4

```
class Point2D implements Comparable<Point2D> {
    private final int x, y;
    public Point2D(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
```

```
    public int getY() { return y; }

    @Override public int compareTo(Point2D o) {
        if (this.x != o.x) return Integer.compare(this.x, o.x);
        return Integer.compare(this.y, o.y);
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Point2D andere = (Point2D) o;
        return x == andere.x && y == andere.y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}
```

Warum Integer.compare und nicht x - o.x? Bei sehr großen Werten kann die Subtraktion überlaufen und ein falsches Vorzeichen liefern. Integer.compare ist immer korrekt.

Lösungen Modul 7 – Lambda

Lösung 7.1

```
Supplier<T>          T get()            nichts   -> T
Consumer<T>          void accept(T)     T        -> nichts
Function<T,R>        R apply(T)         T        -> R
Predicate<T>         boolean test(T)    T        -> boolean
BinaryOperator<T>    T apply(T,T)       T,T      -> T
```

Lösung 7.2

Korrekt: (a, b) -> a + b · (a, b) -> { return a + b; } · x -> x * x · (int a, int b) -> a + b · () -> 42

Falsch: a, b -> a + b (mehrere Parameter ohne Klammern) und (a, b) -> return a + b; (return nur im Block)

Lösung 7.3

alle verifiziert

```
a) Predicate<String> laengerAls5 = s -> s.length() > 5;
b) Function<String,Integer> laenge = s -> s.length();
c) Function<Integer,Integer> quadrat = n -> n * n;
d) Supplier<String> hallo = () -> "Hallo";
e) Consumer<String> drucke = s -> System.out.println(">> " + s);
f) BinaryOperator<Integer> groesser = (a, b) -> a > b ? a : b;
g) UnaryOperator<String> gross = s -> s.toUpperCase();
```

Lösung 7.4

```
static ArrayList<Buch> filtere(ArrayList<Buch> liste, Predicate<Buch> p) {
    ArrayList<Buch> ergebnis = new ArrayList<>();
    for (Buch b : liste) {
        if (p.test(b)) ergebnis.add(b);
    }
    return ergebnis;
}

// Aufrufe:
filtere(buecher, b -> b.getSeiten() > 200);
filtere(buecher, b -> b.getTitel().startsWith("F"));
```

Lösung 7.5

String::length = Instanzmethode eines gegebenen Typs · Integer::parseInt = statisch · ArrayList::new = Konstruktor · System.out::println = Instanzmethode eines bestimmten Objekts

Lösungen Modul 8 – Generics

Lösung 8.1

Erlaubt: Box<Integer> b = new Box<Integer>() · Box b = new Box() (Raw Type, aber Warnung) · Box<String> b = new Box<>() (Diamond)

Nicht erlaubt: Box<int> – primitive Typen gehen nicht, es müsste Box<Integer> heißen.

Lösung 8.2

```
class Paar<K, V> {
    private final K key;
    private final V value;

    public Paar(K key, V value) {
        this.key   = key;
        this.value = value;
    }
    public K getKey()   { return key; }
    public V getValue() { return value; }
}

Paar<String, Integer> p = new Paar<>("Alter", 21);
p.getKey();      // Alter
p.getValue();    // 21
```

Lösungen Modul 9 – Die Verwalterklasse

Lösung 9.1

```
class Buecherregal<T extends Buch> {
    private ArrayList<T> buecher = new ArrayList<>();
    private HashMap<String, T> byTitel = new HashMap<>();

    public void add(T buch) {
        buecher.add(buch);
        byTitel.put(buch.getTitel(), buch);
    }
    public T findByTitel(String titel) {
        return byTitel.get(titel);
    }
    public ArrayList<T> getAll() {
        return new ArrayList<>(buecher);
    }
    public ArrayList<T> filter(Predicate<T> p) {
        ArrayList<T> ergebnis = new ArrayList<>();
        for (T b : buecher) {
            if (p.test(b)) ergebnis.add(b);
        }
        return ergebnis;
    }
    public int gesamtSeiten() {
        int summe = 0;
        for (T b : buecher) summe += b.getSeiten();
        return summe;
    }
}
```

Lösung 9.2

```
class Etage {
    private final int nummer;
    private final ArrayList<Tier> tiere = new ArrayList<>();
```

```
    public Etage(int nummer) { this.nummer = nummer; }
    public void add(Tier t)  { tiere.add(t); }
    public ArrayList<Tier> getTiere() { return tiere; }
}

class Gebaeude {
    private final HashMap<Integer, Etage> etagen = new HashMap<>();

    public void addTier(int nr, Tier t) {
        if (!etagen.containsKey(nr)) {          // Etage anlegen falls noetig
            etagen.put(nr, new Etage(nr));
        }
        etagen.get(nr).add(t);
    }

    public ArrayList<Tier> alleTiere() {
        ArrayList<Tier> alle = new ArrayList<>();
        for (Etage e : etagen.values()) {       // Ebene 1: alle Etagen
            for (Tier t : e.getTiere()) {       // Ebene 2: alle Tiere darin
                alle.add(t);
            }
        }
        return alle;
    }
}
```

Kürzer geht es auch

Statt der if-Abfrage in addTier kannst du eine Zeile schreiben: etagen.computeIfAbsent(nr, k -> new Etage(nr)).add(t);

Und in alleTiere ersetzt in Modul 11 ein flatMap die doppelte Schleife. Beides ist eleganter, beides gibt aber nicht mehr Punkte als die Schleifenversion.

Lösung 9.3

Das ist die vollständige Klausuraufgabe. Vergleiche Teil für Teil.

a) und b)

```
abstract class Fahrzeug implements Comparable<Fahrzeug> {
    private final String name;
    private final int baujahr;

    public Fahrzeug(String name, int baujahr) {
        this.name = name;
        this.baujahr = baujahr;
    }
    public String getName()  { return name; }
    public int getBaujahr()  { return baujahr; }

    public abstract double steuer();

    @Override public int compareTo(Fahrzeug o) {
        return this.name.compareTo(o.name);
    }
    @Override public String toString() {
        return name + " (" + baujahr + ")";
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return name.equals(((Fahrzeug) o).name);
    }
    @Override public int hashCode() { return Objects.hash(name); }
```

```
}

class PKW extends Fahrzeug {
    private final int sitze;
    public PKW(String n, int b, int s) { super(n, b); sitze = s; }
    public int getSitze() { return sitze; }
    @Override public double steuer() { return sitze * 20.0; }
    @Override public String toString() { return "PKW " + super.toString(); }
}

class LKW extends Fahrzeug {
    private final double tonnen;
    public LKW(String n, int b, double t) { super(n, b); tonnen = t; }
    public double getTonnen() { return tonnen; }
    @Override public double steuer() { return tonnen * 30.0; }
    @Override public String toString() { return "LKW " + super.toString(); }
}
```

c) und d) – mit Schleifen

```
class Fuhrpark<T extends Fahrzeug> {
    private final ArrayList<T> items = new ArrayList<>();
    private final HashMap<String, T> byName = new HashMap<>();

    public void add(T f) {
        items.add(f);
        byName.put(f.getName(), f);
    }
    public boolean remove(String n) {
        T t = byName.remove(n);
        return t != null && items.remove(t);
    }
    public T findByName(String n) { return byName.get(n); }
    public ArrayList<T> getAll()  { return new ArrayList<>(items); }

    public ArrayList<T> filter(Predicate<T> p) {
        ArrayList<T> r = new ArrayList<>();
        for (T f : items) if (p.test(f)) r.add(f);
        return r;
    }
    public void applyToAll(Consumer<T> c) {
        for (T f : items) c.accept(f);
    }
    public double gesamtSteuer() {
        double s = 0;
        for (T f : items) s += f.steuer();
        return s;
    }
    public T aeltestes() {
        if (items.isEmpty()) return null;
        T aeltest = items.get(0);
        for (T f : items)
            if (f.getBaujahr() < aeltest.getBaujahr()) aeltest = f;
        return aeltest;
    }
}
```

d) – dieselben Methoden mit Streams (nach Modul 11)

```
public ArrayList<T> filter(Predicate<T> p) {
    return items.stream().filter(p)
                .collect(Collectors.toCollection(ArrayList::new));
}
public void applyToAll(Consumer<T> c) { items.forEach(c); }
public double gesamtSteuer() {
    return items.stream().mapToDouble(Fahrzeug::steuer).sum();
```

```
}
public Optional<T> aeltestes() {
    return items.stream().min(Comparator.comparingInt(Fahrzeug::getBaujahr));
}
```

e) Verifizierte Ausgabe

```
alle          : [PKW VW Golf (2018), PKW Audi A4 (2020),
                 LKW MAN TGX (2015), LKW Volvo FH (2021)]
ab 2018       : [PKW VW Golf (2018), PKW Audi A4 (2020), LKW Volvo FH (2021)]
Steuer gesamt : 1115.0
aeltestes     : LKW MAN TGX (2015)
```

Die erwartete Begründung zu c)

Ich verwende eine ArrayList, damit in Einfügereihenfolge iteriert werden kann, und zusätzlich eine HashMap vom Namen auf das Fahrzeug, weil die Suche über den Namen dadurch in konstanter Zeit möglich ist statt linear. Beide Strukturen werden in add gemeinsam gepflegt.

Lösungen Modul 10 – Collections

Lösung 10.1

```
ArrayList       Duplikate ja    Einfuegereihenfolge
HashSet         nein            keine garantierte Reihenfolge
LinkedHashSet   nein            Einfuegereihenfolge
TreeSet         nein            sortiert
HashMap         Keys eindeutig  keine
TreeMap         Keys eindeutig  nach Key sortiert
ArrayDeque      ja              FIFO bzw. LIFO
PriorityQueue   ja              kleinstes Element zuerst
```

Lösung 10.2

verifiziert

```
ArrayList      -> [3, 1, 3]
HashSet        -> [1, 3]
TreeSet        -> [1, 3]
LinkedHashSet  -> [3, 1]
```

Lösung 10.3

Richtig: List ist ein Interface. ArrayList implementiert List. Ein TreeSet braucht Comparable oder einen Comparator. Eine Queue arbeitet standardmäßig nach FIFO.

Falsch: Map ist KEIN Untertyp von Collection. ArrayList ist NICHT threadsicher. Ein HashSet funktioniert NICHT korrekt ohne hashCode. Eine PriorityQueue liefert das KLEINSTE Element.

Lösung 10.4

```
Produkte ueber ID finden      HashMap<Integer,Product>  konstante Suchzeit
keine doppelten Buecher       HashSet<Book>             Set verbietet Duplikate
sortiert nach Rueckgabedatum  TreeSet mit Comparator    sortiert und ohne Duplikate
in Eingangsreihenfolge        ArrayDeque als Queue      FIFO
nach Prioritaet               PriorityQueue + Comparator kleinstes zuerst
in Einfuegereihenfolge        LinkedHashMap / ArrayList behaelt die Reihenfolge
Quersumme -> Zeitstempel      Map<Integer,List<Long>>   mehrere Werte je Schluessel
```

Lösung 10.5

```
a) map.getOrDefault("a", 0);
b) map.putIfAbsent(k, v);
```

```
c) for (Map.Entry<String,Integer> e : map.entrySet())
       System.out.println(e.getKey() + " = " + e.getValue());
d) map.forEach((k, v) -> System.out.println(k + " = " + v));
e) map.computeIfAbsent(k, key -> new ArrayList<>()).add(x);
```

Lösung 10.6

```
class Product {
    private final int productId;
    private final String name, category;
    private final double price;
    private final int quantity;
    public Product(int id, String n, String c, double p, int q) {
        productId = id; name = n; category = c; price = p; quantity = q;
    }
    public int getProductId()   { return productId; }
    public String getName()     { return name; }
    public String getCategory() { return category; }
    public double getPrice()    { return price; }
    public int getQuantity()    { return quantity; }
}

class Inventory {
    private final Map<Integer, Product> byId = new LinkedHashMap<>();

    public void addProduct(Product p) { byId.put(p.getProductId(), p); }
    public boolean removeProduct(int id) { return byId.remove(id) != null; }
    public Product findProductById(int id) { return byId.get(id); }

    public List<Product> getAllProducts() { return new ArrayList<>(byId.values()); }

    public List<Product> findProductsByCategory(String c) {
        return byId.values().stream()
                   .filter(p -> p.getCategory().equals(c))
                   .collect(Collectors.toList());
    }
    public List<Product> sortProductsByPrice() {
        return byId.values().stream()
                   .sorted(Comparator.comparingDouble(Product::getPrice))
                   .collect(Collectors.toList());
    }
    public List<Product> getLowStockProducts(int threshold) {
        return byId.values().stream()
                   .filter(p -> p.getQuantity() < threshold)
                   .collect(Collectors.toList());
    }
    public List<Product> filterProducts(Predicate<Product> p) {
        return byId.values().stream().filter(p).collect(Collectors.toList());
    }
    public void applyToProducts(Consumer<Product> c) { byId.values().forEach(c); }
}
```

Die erwartete Begründung

Eine LinkedHashMap erfüllt beide Anforderungen gleichzeitig: Über den Schlüssel productId findet und entfernt sie Produkte in konstanter Zeit, und im Gegensatz zur HashMap behält sie die Einfügereihenfolge bei – damit ist auch Teilaufgabe b erfüllt, ohne eine zweite Struktur pflegen zu müssen.

Lösung 10.7

```
private final Map<Integer, List<Long>> zeitstempel = new TreeMap<>();

public void consume(int zahl) {
    int qs = quersumme(zahl);
```

```
    zeitstempel.computeIfAbsent(qs, k -> new ArrayList<>())
               .add(System.currentTimeMillis());
}
public int numberOfOccurrences(int quersumme) {
    return zeitstempel.getOrDefault(quersumme, List.of()).size();
}
public Collection<Integer> getCrossTotalsAscending() {
    return zeitstempel.keySet();      // TreeMap -> bereits aufsteigend
}
```

Eine TreeMap ist hier besser als eine HashMap, weil die Quersummen aufsteigend ausgegeben werden sollen – keySet() liefert sie dann bereits sortiert.

Lösungen Modul 11 – Streams

Lösung 11.1

Intermediate: filter, map, sorted, distinct, limit, flatMap. Terminal: forEach, reduce, collect, count, anyMatch, findFirst.

Regel: Alles, was wieder einen Stream zurückgibt, ist intermediate. Alles, was einen Wert, eine Collection oder nichts zurückgibt, ist terminal.

Lösung 11.2

Richtig: Ohne Terminal-Operation passiert nichts. map gibt einen Stream zurück.

Falsch: Ein Stream speichert keine Daten, verändert die Quelle nicht, kann nur einmal verwendet werden, und collect gibt keinen Stream zurück.

Lösung 11.3

alle verifiziert

```
a) s.stream().count()                                        -> 6
b) s.stream().map(Student::name).toList()
c) s.stream().filter(x -> x.note() < 5).map(Student::name).toList()
d) s.stream().mapToInt(Student::note).average().getAsDouble()  -> 2.83
e) s.stream().min(Comparator.comparingInt(Student::note)).get().name() -> Clara
f) s.stream().sorted(Comparator.comparingInt(Student::note))
            .map(Student::name).toList()
g) s.stream().mapToInt(Student::semester).sum()              -> 20
h) s.stream().map(Student::kurs).distinct().toList()        -> [Info, Mathe, BWL]
i) s.stream().map(Student::name).collect(Collectors.joining(", "))
j) s.stream().allMatch(x -> x.note() < 5)                   -> false
```

Lösung 11.4

verifiziert

```
a) 6      Startwert 1: 1*1=1, 1*2=2, 2*3=6
b) 6      Startwert 0: 0+1+2+3
c) [1, 3, 4]
d) true
e) 6      1 + 2 + 3
```

Lösung 11.5

alle verifiziert

```
a) s.stream().collect(Collectors.groupingBy(Student::kurs,
                     Collectors.counting()))
   -> {BWL=1, Mathe=2, Info=3}

b) s.stream().collect(Collectors.groupingBy(Student::kurs,
                     Collectors.averagingInt(Student::note)))
```

```
   -> {BWL=5.0, Mathe=1.5, Info=3.0}

c) s.stream().collect(Collectors.groupingBy(Student::kurs,
       Collectors.collectingAndThen(
           Collectors.minBy(Comparator.comparingInt(Student::note)),
           o -> o.get().name())))
   -> {BWL=Felix, Mathe=Clara, Info=Anna}

d) s.stream().collect(Collectors.partitioningBy(x -> x.note() < 5,
                     Collectors.counting()))
   -> {false=1, true=5}

e) s.stream().collect(Collectors.groupingBy(Student::kurs, Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String,Long>comparingByValue().reversed())
    .toList()
   -> [Info=3, Mathe=2, BWL=1]

f) s.stream().collect(Collectors.groupingBy(
       x -> x.note() <= 2 ? "gut" : x.note() <= 4 ? "mittel" : "schlecht",
       Collectors.counting()))
```

Lösung 11.6

```
int gesamtSeiten() {
    return buecher.stream().mapToInt(Book::getPages).sum();
}
List<Book> abJahr(int jahr) {
    return buecher.stream().filter(b -> b.getYear() > jahr).toList();
}
List<Book> nachGenre(String g) {
    return buecher.stream().filter(b -> b.getGenre().equals(g)).toList();
}
Map<String, Double> bewertungJeGenre() {
    return buecher.stream().collect(Collectors.groupingBy(
        Book::getGenre, Collectors.averagingDouble(Book::getRating)));
}
List<Book> topDrei() {
    return buecher.stream()
        .sorted(Comparator.comparingDouble(Book::getRating).reversed())
        .limit(3).toList();
}
List<Map.Entry<String,Long>> autorenNachAnzahl() {
    return buecher.stream()
        .collect(Collectors.groupingBy(Book::getAuthor, Collectors.counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String,Long>comparingByValue().reversed())
        .toList();
}
List<Book> nachSeiten() {
    return buecher.stream()
        .sorted(Comparator.comparingInt(Book::getPages)).toList();
}
List<Book> gefiltertSortiert(Predicate<Book> p, Comparator<Book> c) {
    return buecher.stream().filter(p).sorted(c).toList();
}
```

Lösungen Modul 12 – Innere Klassen

Lösung 12.1

```
statisch  in der Klasse mit static   nur statische Felder    ohne aeusseres Objekt
echt      in der Klasse ohne static  alle Felder auch private braucht outer.new Inner()
lokal     in einer Methode           Felder + lokale Vars    nur in der Methode nutzbar
```

```
          (wenn effektiv final)
anonym    direkt bei new             wie lokale Klasse       kein Name, einmalig
```

Lösung 12.2

```
new Outer.StatischInnere();      // ohne Outer-Objekt
new Outer().new EchtInnere();    // mit Outer-Objekt
```

Lösung 12.3

verifiziert

```
i: 2      der Parameter
i: 1      das Feld der inneren Klasse
i: 0      das statische Feld der aeusseren Klasse
```

Lösung 12.4

```
interface BinaryOperation { int apply(int a, int b); }

class Calculator {
    public int compute(int a, int b, BinaryOperation op) {
        return op.apply(a, b);
    }
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.compute(6, 3, new BinaryOperation() {
            @Override public int apply(int a, int b) { return a + b; } }));
        System.out.println(c.compute(6, 3, new BinaryOperation() {
            @Override public int apply(int a, int b) { return a - b; } }));
        System.out.println(c.compute(6, 3, new BinaryOperation() {
            @Override public int apply(int a, int b) { return a * b; } }));
        System.out.println(c.compute(6, 3, new BinaryOperation() {
            @Override public int apply(int a, int b) { return a / b; } }));
    }
}
```

Lösung 12.5

```
public class Lager {
    private Fach[] faecher;

    public interface Iterator {
        boolean hasNext();
        Fach next();
    }

    private class LagerIterator implements Iterator {
        private final String typ;
        private int pos = 0;
        private LagerIterator(String typ) { this.typ = typ; }

        @Override public boolean hasNext() {
            while (pos < faecher.length && !faecher[pos].getTyp().equals(typ)) pos++;
            return pos < faecher.length;
        }
        @Override public Fach next() {
            if (!hasNext()) throw new NoSuchElementException();
            return faecher[pos++];
        }
    }

    public Iterator iterator(String typ) { return new LagerIterator(typ); }
```

```
}
```

Erwartete Begründung: Der Iterator ist nur im Kontext des Lagers sinnvoll und kann als innere Klasse direkt auf dessen privates Array zugreifen, ohne dass dieses nach außen geöffnet werden muss. Die Kapselung bleibt damit erhalten.

Lösungen Modul 13 – Rekursion

Lösung 13.1

Eine Rekursion endet, wenn der Basisfall erreicht ist. Die Aufrufe werden auf dem Stack gespeichert. Bei unendlicher Rekursion läuft der Stack über und es kommt zum StackOverflowError. Rekursion ist langsamer, weil für jeden Aufruf ein Stack-Frame angelegt und abgebaut werden muss und bei naiver Rekursion Teilprobleme mehrfach berechnet werden.

Lösung 13.2

```
if (Basisfall) return Basiswert;       // 1. Abbruch zuerst
// 2. Problem verkleinern
return kombiniere(methode(kleineresProblem));   // 3. rekursiver Aufruf
```

Lösung 13.3

alle verifiziert

```
a) static int summe(int n) {
       if (n <= 0) return 0;
       return n + summe(n - 1);          // summe(5) -> 15
   }
b) static int anzahlZiffern(int n) {
       if (n < 10) return 1;
       return 1 + anzahlZiffern(n / 10); // 4711 -> 4
   }
c) static int quersumme(int n) {
       if (n == 0) return 0;
       return n % 10 + quersumme(n / 10); // 1234 -> 10
   }
d) static int potenz(int b, int e) {
       if (e == 0) return 1;
       return b * potenz(b, e - 1);      // 2^10 -> 1024
   }
e) static int ggT(int a, int b) {
       if (b == 0) return a;
       return ggT(b, a % b);             // ggT(48,18) -> 6
   }
f) static boolean istPalindrom(String s, int l, int r) {
       if (l >= r) return true;
       if (s.charAt(l) != s.charAt(r)) return false;
       return istPalindrom(s, l + 1, r - 1);
   }
g) static int maxArray(int[] a, int i) {
       if (i == a.length - 1) return a[i];
       return Math.max(a[i], maxArray(a, i + 1));  // {3,9,2,7} -> 9
   }
```

Lösung 13.4

```
public static void bewege(int i, char a, char b, char c) {
    if (i > 0) {
        bewege(i - 1, a, c, b);
        System.out.println("Scheibe von " + a + " nach " + c);
        bewege(i - 1, b, a, c);
    }
}
```

Erwartete Erklärung: Um n Scheiben von a nach c zu bewegen, verschiebt man zuerst die oberen n−1 Scheiben auf den Hilfsstab b. Damit liegt die größte Scheibe frei und kann direkt nach c gelegt werden. Anschließend werden die n−1 Scheiben von b nach c bewegt. Beide Teilprobleme sind strukturgleich, nur kleiner – deshalb greift Rekursion. Der Basisfall n = 0 tut nichts.

Anzahl der Züge: 2 hoch n minus 1.

Lösung 13.5

```
public interface Palindrom { boolean istPalindrom(String wort); }

public class PalindromRekursiv implements Palindrom {
    @Override public boolean istPalindrom(String w) {
        return pruefe(w, 0, w.length() - 1);
    }
    private boolean pruefe(String w, int l, int r) {
        if (l >= r) return true;
        if (w.charAt(l) != w.charAt(r)) return false;
        return pruefe(w, l + 1, r - 1);
    }
}

public class PalindromIterativ implements Palindrom {
    @Override public boolean istPalindrom(String w) {
        int l = 0, r = w.length() - 1;
        while (l < r) {
            if (w.charAt(l) != w.charAt(r)) return false;
            l++; r--;
        }
        return true;
    }
}
```

Erklärung: Beide Varianten haben lineare Laufzeit, aber die rekursive legt für jeden Vergleich einen eigenen Stack-Frame an und baut ihn wieder ab. Dieser Verwaltungsaufwand fällt bei langen Wörtern deutlich ins Gewicht und macht die rekursive Variante messbar langsamer.

Lösungen Modul 14 – C: Pointer

Lösung 14.1

& heißt Adress-Operator und liefert die Adresse einer Variablen. * heißt Dereferenzierungs-Operator und liefert den Wert, auf den ein Zeiger zeigt. &*p ist dasselbe wie p – die beiden Operatoren heben sich auf.

Lösung 14.2

verifiziert

```
*p            -> 10        p[3]        -> 40
*(p + 3)      -> 40        *(p) + 3    -> 13
p[0]          -> 10        *(p + 4)    -> 50
nach p += 2   -> 30        a == &a[0]  -> wahr (1)
```

*(p+3) verschiebt zuerst den Zeiger um drei Elemente und liest dann den Wert. *(p)+3 liest zuerst den Wert an der aktuellen Position und addiert dann 3.

Lösung 14.3

Die Adresse von l ist die erste Ausgabe, 0x7ffee9ca2b38, denn &*pl ist die Adresse von l.

Zutreffend sind die ersten vier Aussagen. Falsch ist nur „Alle drei Ausgaben sind identisch".

Lösung 14.4

alle verifiziert

```
a) void verdopple(int* x) { *x = *x * 2; }
```

```
b) void tausche(int* a, int* b) {
       int t = *a; *a = *b; *b = t;
   }

c) int summeArray(int* a, int n) {
       int s = 0;
       for (int i = 0; i < n; i++) s += a[i];
       return s;
   }

d) void minMax(int* a, int n, int* mn, int* mx) {
       *mn = a[0]; *mx = a[0];
       for (int i = 1; i < n; i++) {
           if (a[i] < *mn) *mn = a[i];
           if (a[i] > *mx) *mx = a[i];
       }
   }
// Aufruf: int mn, mx; minMax(arr, 6, &mn, &mx);
```

Lösung 14.5

verifiziert

```
5
6
```

Der erste Aufruf ändert nur die lokale Kopie, weil C immer call by value übergibt. Erst über den Pointer wird die Originalvariable verändert.

Lösung 14.6

alle verifiziert mit dem Array {4,9,2,7,1,8}

```
a) void summeRek(int* a, int n, int i, int* s) {
       if (i >= n) return;
       *s += a[i];
       summeRek(a, n, i + 1, s);       // -> 31
   }
b) void maxRek(int* a, int n, int i, int* m) {
       if (i >= n) return;
       if (a[i] > *m) *m = a[i];
       maxRek(a, n, i + 1, m);         // -> 9
   }
c) void zaehleGerade(int* a, int n, int i, int* c) {
       if (i >= n) return;
       if (a[i] % 2 == 0) (*c)++;
       zaehleGerade(a, n, i + 1, c);   // -> 3
   }
```

Achtung bei c): Die Klammern in (*c)++ sind nötig. Ohne sie würde *c++ zuerst den Zeiger erhöhen statt den Wert.

Lösung 14.7

verifiziert

```
int isPalindrome(const char* s, int left, int right) {
    if (left >= right)       return 1;
    if (s[left] != s[right]) return 0;
    return isPalindrome(s, left + 1, right - 1);
}

int isPalindromeIter(const char* s) {
    int l = 0, r = strlen(s) - 1;
    while (l < r) {
        if (s[l] != s[r]) return 0;
```

```
        l++; r--;
    }
    return 1;
}

// otto -> 1    reittier -> 1    hallo -> 0
```

Lösung 14.8

```
void findMin(int* input, int size, int index, int* minNumber) {
    if (index >= size) return;
    if (input[index] < *minNumber) *minNumber = input[index];
    findMin(input, size, index + 1, minNumber);
}

int main(void) {
    int arr[6] = {42, 7, 19, 3, 88, 15};
    int min = arr[0];              // WICHTIG: mit dem ersten Element
    findMin(arr, 6, 0, &min);
    printf("min = %d\n", min);      // -> min = 3
    return 0;
}

// d) mit Index:
void findMin2(int* in, int size, int i, int* min, int* idx) {
    if (i >= size) return;
    if (in[i] < *min) { *min = in[i]; *idx = i; }
    findMin2(in, size, i + 1, min, idx);
}
```

Zu b): minNumber muss mit dem ersten Arrayelement initialisiert werden, nicht mit 0. Bei 0 würde bei ausschließlich positiven Zahlen immer 0 als Minimum herauskommen.

Zu c): C übergibt Parameter immer call by value, die Funktion arbeitet also nur mit Kopien. Da die Funktion void ist und außerdem mehrere Werte zurückgeben können soll, muss das Ergebnis über die Adresse geschrieben werden. Nur so ist die Änderung außerhalb der Funktion sichtbar.

Lösungen Modul 15 – C: Structs und Matrizen

Lösung 15.1

Strukturen definiert man mit struct. typedef vergibt zusätzlich einen kurzen Typnamen, sodass man nicht jedes Mal struct mitschreiben muss.

Punkt beim Zugriff über das Objekt, Pfeil beim Zugriff über einen Pointer.

Bei einem struct hat jedes Feld eigenen Speicher, die Größe ist die Summe aller Felder. Bei einer union teilen sich alle Felder denselben Speicher, die Größe ist die des größten Feldes – es ist immer nur ein Feld gültig.

Große structs werden als Pointer übergeben, weil sonst die gesamte Struktur kopiert würde. Mit const signalisiert man zusätzlich, dass die Funktion nur liest.

Lösung 15.2

```
matrix[i][j]   entspricht   m[i * cols + j]
```

Weil ein zweidimensionales Array im Speicher zeilenweise hintereinander liegt, kann es als flacher Block behandelt werden. Innerhalb einer Funktion ist die Spaltenzahl aber nicht mehr bekannt – sizeof liefert dort nur noch die Größe des Zeigers. Deshalb müssen rows und cols als eigene Parameter übergeben werden.

Lösung 15.3

verifiziert

```
typedef struct { char name[20]; int alter; float note; } Student;
```

```
void printStudent(const Student* s) {
    printf("%s, %d J., Note %.1f\n", s->name, s->alter, s->note);
}
float schnitt(Student* arr, int n) {
    float s = 0;
    for (int i = 0; i < n; i++) s += arr[i].note;
    return s / n;
}
int findeIndex(Student* arr, int n, const char* name) {
    for (int i = 0; i < n; i++)
        if (strcmp(arr[i].name, name) == 0) return i;
    return -1;
}
```

Lösung 15.4

verifiziert

```
a) int summeZeile(int* m, int rows, int cols, int row) {
       int s = 0;
       for (int j = 0; j < cols; j++) s += m[row * cols + j];
       return s;
   }
b) int maxSpalte(int* m, int rows, int cols, int col) {
       int mx = m[col];
       for (int i = 1; i < rows; i++)
           if (m[i * cols + col] > mx) mx = m[i * cols + col];
       return mx;
   }
c) void swapRows(int* m, int rows, int cols, int r1, int r2) {
       for (int j = 0; j < cols; j++) {
           int t = m[r1 * cols + j];
           m[r1 * cols + j] = m[r2 * cols + j];
           m[r2 * cols + j] = t;
       }
   }
d) void transpose(int* m, int n) {
       for (int i = 0; i < n; i++)
           for (int j = i + 1; j < n; j++) {   // j startet bei i+1!
               int t = m[i * n + j];
               m[i * n + j] = m[j * n + i];
               m[j * n + i] = t;
           }
   }
```

Bei d) ist entscheidend, dass die innere Schleife bei j = i+1 startet. Startet sie bei 0, wird jedes Paar zweimal getauscht und die Matrix bleibt unverändert.

Lösung 15.5

```
#define MAX 50
typedef struct {
    char name[20];
    int  alter;
    char abteilung[30];
    float gehalt;
} Mitarbeiter;

Mitarbeiter liste[MAX];
int anzahl = 0;

void addMitarbeiter(void) {
    if (anzahl >= MAX) { printf("Array voll\n"); return; }
    printf("Name: ");      scanf("%s",  liste[anzahl].name);
```

```
    printf("Alter: ");     scanf("%d", &liste[anzahl].alter);
    printf("Abteilung: "); scanf("%s",  liste[anzahl].abteilung);
    printf("Gehalt: ");    scanf("%f", &liste[anzahl].gehalt);
    anzahl++;
}
int findeIndex(const char* name) {
    for (int i = 0; i < anzahl; i++)
        if (strcmp(liste[i].name, name) == 0) return i;
    return -1;
}
void deleteMitarbeiter(char* name) {
    int i = findeIndex(name);
    if (i < 0) { printf("nicht gefunden\n"); return; }
    for (int k = i; k < anzahl - 1; k++) liste[k] = liste[k + 1];
    anzahl--;
}
void queryMitarbeiter(char* name) {
    int i = findeIndex(name);
    if (i < 0) { printf("nicht gefunden\n"); return; }
    printf("%s, %d, %s, %.2f\n", liste[i].name, liste[i].alter,
           liste[i].abteilung, liste[i].gehalt);
}
void printAllMitarbeiter(void) {
    for (int i = 0; i < anzahl; i++) queryMitarbeiter(liste[i].name);
}
```

Wichtig: Bei scanf mit %s steht kein &, weil der Array-Name schon die Adresse ist. Bei %d und %f dagegen schon. Beim Löschen rücken alle folgenden Elemente auf, damit kein Loch entsteht.

Lösungen Modul 16 – Code-Ausgaben

Lösung 16.1

verifiziert

```
B
6
```

Das B wird nur im Basisfall gedruckt, also einmal bei fac(1). Danach: 1, dann 2·1 = 2, dann 3·2 = 6.

Lösung 16.2

verifiziert

```
vor 3
vor 2
vor 1
nach 1
nach 2
nach 3
```

Die Ausgaben vor dem rekursiven Aufruf laufen abwärts, die danach beim Zurückkehren aufwärts. Das ist der klassische Aufbau- und Abbau-Effekt des Stacks.

Lösung 16.3

```
48
```

Startwert 2: 2·1 = 2, 2·2 = 4, 4·3 = 12, 12·4 = 48.

Lösung 16.4

```
[ANNA, CLARA]
```

Ben hat nur drei Buchstaben und fällt beim Filter heraus.

Lösung 16.5

```
3 [1, 3, 5] [5, 3, 1]
```

Alle drei Mengen enthalten drei Elemente. TreeSet sortiert aufsteigend, LinkedHashSet behält die Einfügereihenfolge.

Lösung 16.6

```
40
31
```

andThen: (3+1)·10 = 40. compose: 3·10 = 30, dann +1 = 31.

Lösung 16.7

```
{a=1, b=5}
0
```

Das zweite put auf b überschreibt den alten Wert. Die TreeMap sortiert nach Schlüssel. getOrDefault liefert für einen fehlenden Schlüssel den Standardwert.

Lösung 16.8

```
4 8 6
```

Nach p++ zeigt p auf die 4. *(p+2) ist die 8. *(p)+2 ist 4+2 = 6.

Lösung 16.9

```
3
3.50
1
```

7/2 ist eine Ganzzahldivision und ergibt 3. Erst der Cast auf double liefert 3.50. Der Modulo-Rest ist 1.

Lösung 16.10

```
1
1
```

Die Queue arbeitet nach FIFO und liefert die zuerst eingefügte 1. Die PriorityQueue liefert das kleinste Element, ebenfalls die 1.

Lösung 16.11

```
4 66 B
```

Die union ist so groß wie ihr größtes Feld, also 4 Byte. Der Wert 66 entspricht als Zeichen dem Großbuchstaben B.

Lösung 16.12

```
{false=3, true=3}
```

partitioningBy teilt immer in genau zwei Gruppen unter den Schlüsseln false und true.

Lösungen Modul 17 – Probeklausur

Aufgabe 1 – Multiple Choice

```
1.1   Basisfall erreicht  +  Bedingung wird false
1.2   Im Stack
1.3   Zugriff auf private Felder  +  static oder non-static moeglich
1.4   lokale Variablen (effektiv final)  +  Parameter (effektiv final)
      +  statische Variablen anderer Klassen
1.5   kein Klassenname  +  direkt bei der Objekterzeugung  +  nur einmal
1.6   Compilezeit-Pruefung  +  Type Erasure  +  weniger Casting  +  Wildcards
```

```
1.7   Box<Integer>...  +  Box b = new Box()  +  Box<String> mit Diamond
1.8   (a, b) -> a + b  +  (a, b) -> { return a + b; }  +  (int a, int b) -> a + b
1.9   Consumer<T>
1.10  Interface  +  erlaubt Duplikate  +  Einfuegereihenfolge
1.11  forEach()  +  reduce()  +  collect()
1.12  prozedural  +  direkt zu Maschinencode  +  Zeigerarithmetik
1.13  Verknuepft Objektdateien zu einem Programm
1.14  struct
1.15  Schluessel sortiert  +  Comparable oder Comparator noetig
```

Zu 1.6: „primitive Typen wie int direkt verwenden" ist FALSCH – genau das geht nicht.

Zu 1.15: TreeMap ist kein Untertyp von Collection und erlaubt keine doppelten Schlüssel.

Aufgabe 2 – Wissensfragen

a) Ein funktionales Interface ist ein Interface mit genau einer abstrakten Methode. Dadurch kann es durch einen Lambda-Ausdruck oder eine Methodenreferenz implementiert werden. Zusätzliche default- und static-Methoden sind erlaubt.

b) Für jeden Aufruf wird ein neuer Stack-Frame angelegt. Da der Stack begrenzt ist, läuft er irgendwann über und die JVM wirft einen StackOverflowError.

c) HashSet und HashMap bestimmen über den hashCode zuerst den Speicherbereich, in dem sie suchen. Wenn zwei laut equals gleiche Objekte unterschiedliche hashCodes haben, landen sie in verschiedenen Bereichen und werden nicht als gleich erkannt.

d) Der Adress-Operator & liefert die Adresse einer Variablen, der Stern-Operator * dereferenziert einen Zeiger und liefert den Wert, auf den er zeigt.

e) Type Erasure bedeutet, dass der Compiler die generische Typinformation nach der Typprüfung entfernt, damit der Bytecode kompatibel zu älteren Java-Versionen bleibt. Deshalb sind new T(), Arrays von Typparametern und instanceof mit konkretem Typparameter nicht möglich.

```
f) 6
g) 30
h) [1, 2, 4]
```

Aufgabe 3 – Java-Programmierung

```
abstract class Medium implements Comparable<Medium> {
    private final String titel;
    private final int laenge;

    public Medium(String titel, int laenge) {
        this.titel = titel;
        this.laenge = laenge;
    }
    public String getTitel() { return titel; }
    public int getLaenge()   { return laenge; }

    public abstract double gebuehr();

    @Override public int compareTo(Medium o) {
        return this.titel.compareTo(o.titel);
    }
    @Override public String toString() {
        return titel + " (" + laenge + "s)";
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return titel.equals(((Medium) o).titel);
    }
    @Override public int hashCode() { return Objects.hash(titel); }
}
```

```
class Song extends Medium {
    private final String kuenstler;
    public Song(String t, int l, String k) { super(t, l); kuenstler = k; }
    public String getKuenstler() { return kuenstler; }
    @Override public double gebuehr() { return getLaenge() * 0.01; }
}

class Hoerbuch extends Medium {
    private final String autor;
    public Hoerbuch(String t, int l, String a) { super(t, l); autor = a; }
    public String getAutor() { return autor; }
    @Override public double gebuehr() { return getLaenge() * 0.005; }
}

class Bibliothek<T extends Medium> {
    private final List<T> medien = new ArrayList<>();
    private final Map<String, T> byTitel = new HashMap<>();

    public void add(T m) {
        medien.add(m);
        byTitel.put(m.getTitel(), m);
    }
    public T findByTitel(String t) { return byTitel.get(t); }
    public List<T> getAll() { return new ArrayList<>(medien); }

    public List<T> filter(Predicate<T> p) {
        return medien.stream().filter(p).collect(Collectors.toList());
    }
    public List<T> sortedBy(Comparator<T> c) {
        return medien.stream().sorted(c).collect(Collectors.toList());
    }
    public double gesamtGebuehr() {
        return medien.stream().mapToDouble(Medium::gebuehr).sum();
    }
    public Map<String, List<T>> gruppiereNachTyp() {
        return medien.stream()
            .collect(Collectors.groupingBy(m -> m.getClass().getSimpleName()));
    }
}
```

Erwartete Begründung zu c)

Ich verwende eine ArrayList, um in Einfügereihenfolge iterieren zu können, und zusätzlich eine HashMap vom Titel auf das Medium, weil die Suche über den Titel damit in konstanter Zeit statt linear möglich ist. Beide Strukturen werden in add gemeinsam gepflegt.

Aufgabe 4 – Streams und C

Teil 1

```
songs.stream().count()
songs.stream().mapToInt(Song::getLaenge).average().orElse(0)
songs.stream().filter(s -> s.getGenre().equals("Rock"))
              .map(Song::getTitel).toList()
songs.stream().filter(s -> s.getGenre().equals(g))
              .mapToInt(Song::getLaenge).sum() / 60.0
songs.stream().collect(Collectors.groupingBy(Song::getKuenstler,
                       Collectors.counting()))
songs.stream().collect(Collectors.groupingBy(Song::getKuenstler,
                       Collectors.averagingDouble(Song::getBewertung)))
songs.stream().sorted(Comparator.comparingInt(Song::getAbrufe).reversed())
              .limit(3).toList()
```

```
songs.stream().collect(Collectors.groupingBy(Song::getGenre,
       Collectors.collectingAndThen(
           Collectors.maxBy(Comparator.comparingInt(Song::getLaenge)),
           Optional::get)))
songs.stream().filter(s -> s.getBewertung() > 4.5)
              .map(Song::getKuenstler).distinct().toList()
songs.stream().collect(Collectors.groupingBy(Song::getGenre, Collectors.counting()))
     .entrySet().stream()
     .sorted(Map.Entry.<String,Long>comparingByValue().reversed()).toList()
```

Teil 2

```
void zaehleGroesser(int* a, int n, int i, int grenze, int* anzahl) {
    if (i >= n) return;
    if (a[i] > grenze) (*anzahl)++;
    zaehleGroesser(a, n, i + 1, grenze, anzahl);
}

int main(void) {
    int arr[6] = {4, 9, 2, 7, 1, 8};
    int anzahl = 0;                    // mit 0 initialisieren!
    zaehleGroesser(arr, 6, 0, 5, &anzahl);
    printf("%d\n", anzahl);             // -> 3
    return 0;
}

typedef struct { int x; int y; } Punkt;

double abstand(Punkt a, Punkt b, Punkt* diff) {
    diff->x = b.x - a.x;
    diff->y = b.y - a.y;
    return sqrt(pow(diff->x, 2) + pow(diff->y, 2));
}
```

Nach der Probeklausur

Zähl nicht die Fehler, sondern sortiere sie: Welche kamen aus Unwissen, welche aus Flüchtigkeit? Unwissen behebst du mit dem passenden Modul. Flüchtigkeit behebst du, indem du in der echten Klausur die letzten fünf Minuten zum Durchsehen einplanst.

Und falls Aufgabe 3 gehakt hat: Die schreibst du nochmal komplett neu, am besten am nächsten Tag. Das ist der größte Einzelblock der Klausur.